Intuition Engine Architecture

Last modified: 2026-08-05

Intuition Engine is a multi-CPU fantasy computer with 6 heterogeneous CPU cores, 6 video systems, audio engines and players, a copper coprocessor, DMA blitter, and extensive I/O peripherals - all connected through a unified MachineBus. Total guest RAM is sized at boot from platform-dispatched usable-RAM detection (`/proc/meminfo` on Linux, `GlobalMemoryStatusEx` on Windows, and `hw.memsize` on Darwin) minus a per-platform reserve. Darwin RAM sizing uses a page-aligned conservative half of `hw.memsize` as the detected base before applying the per-platform reserve. Each CPU/profile sees an active visible RAM clamped to its own ceiling. Guest software discovers sizes through the SYSINFO MMIO pairs (`SYSINFO_TOTAL_RAM_LO/HI`, `SYSINFO_ACTIVE_RAM_LO/HI`) and IE64 `CR_RAM_SIZE_BYTES`. This document describes the system architecture with diagrams showing chips, buses, internal functional units, and data flow paths.

The diagrams below describe wired runtime behaviour. Platform availability follows the runtime dispatch path; for example, the Z80 JIT is available on amd64 builds only.

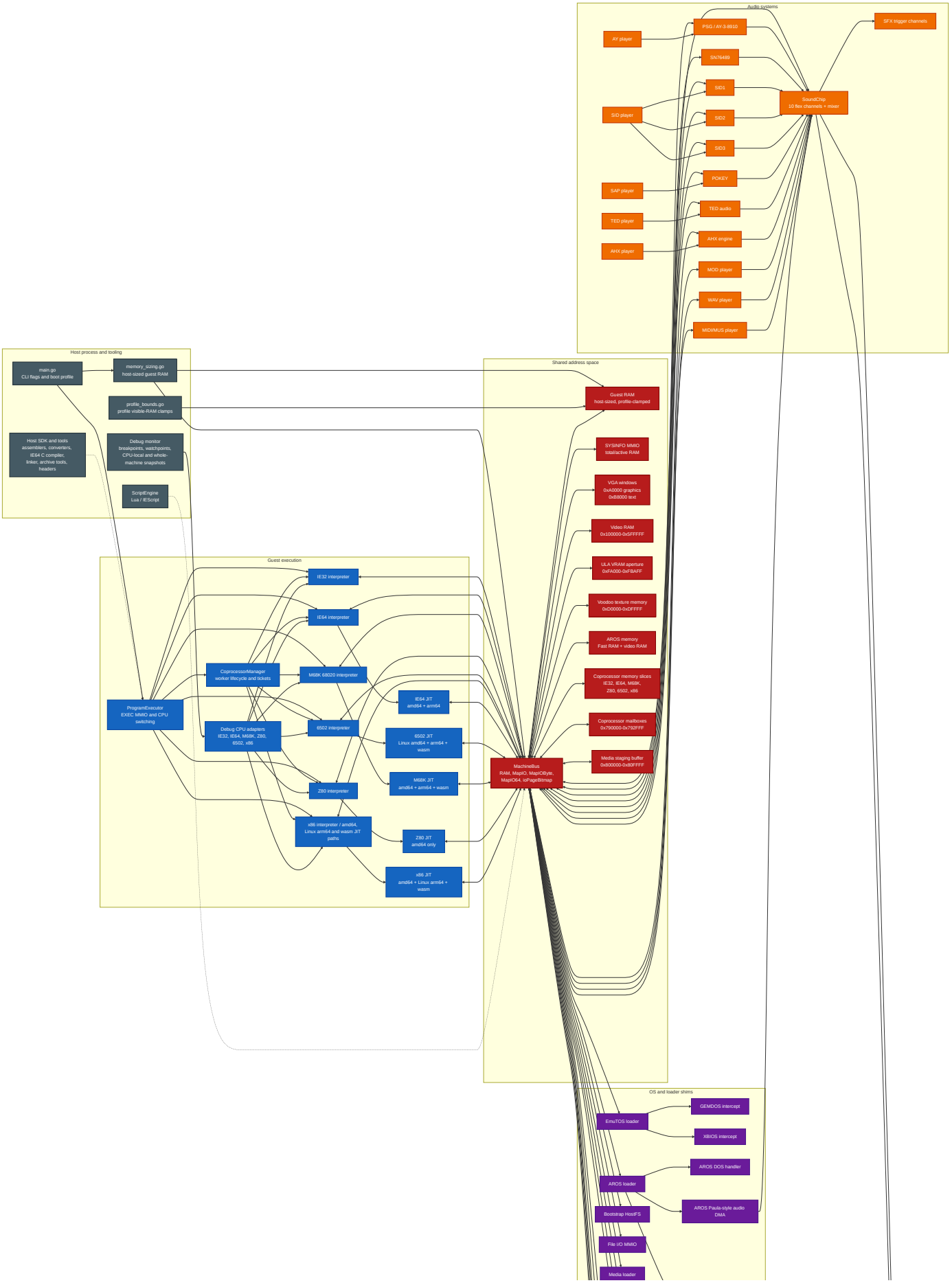
Reading the Architecture Tables and Diagrams

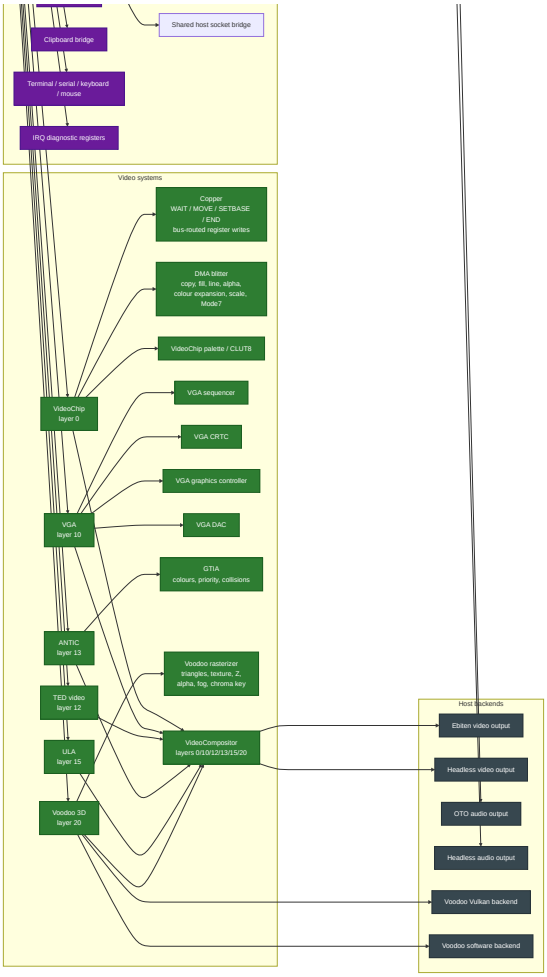
Boxes represent runtime components. Arrows represent operational paths: direct calls, bus mappings, shared-memory paths, queues, or host-service dependencies. Memory-map rows describe guest-visible address ranges and the device or subsystem that owns each range.

Diagram/table item	Meaning
Address range	Guest-visible range plus decoded mapping, owner, and reservation/use semantics
Visible RAM size	RAM visible to the current CPU/profile, as reported by SYSINFO and CPU-specific discovery paths
CPU or JIT entry	CPU core or JIT backend available through the runtime dispatch path
Audio, video, or peripheral box	Engine, player, device, or bridge that participates in the running machine
Debugger or scripting path	Monitor, debug adapter, or Lua binding path used for inspection or automation

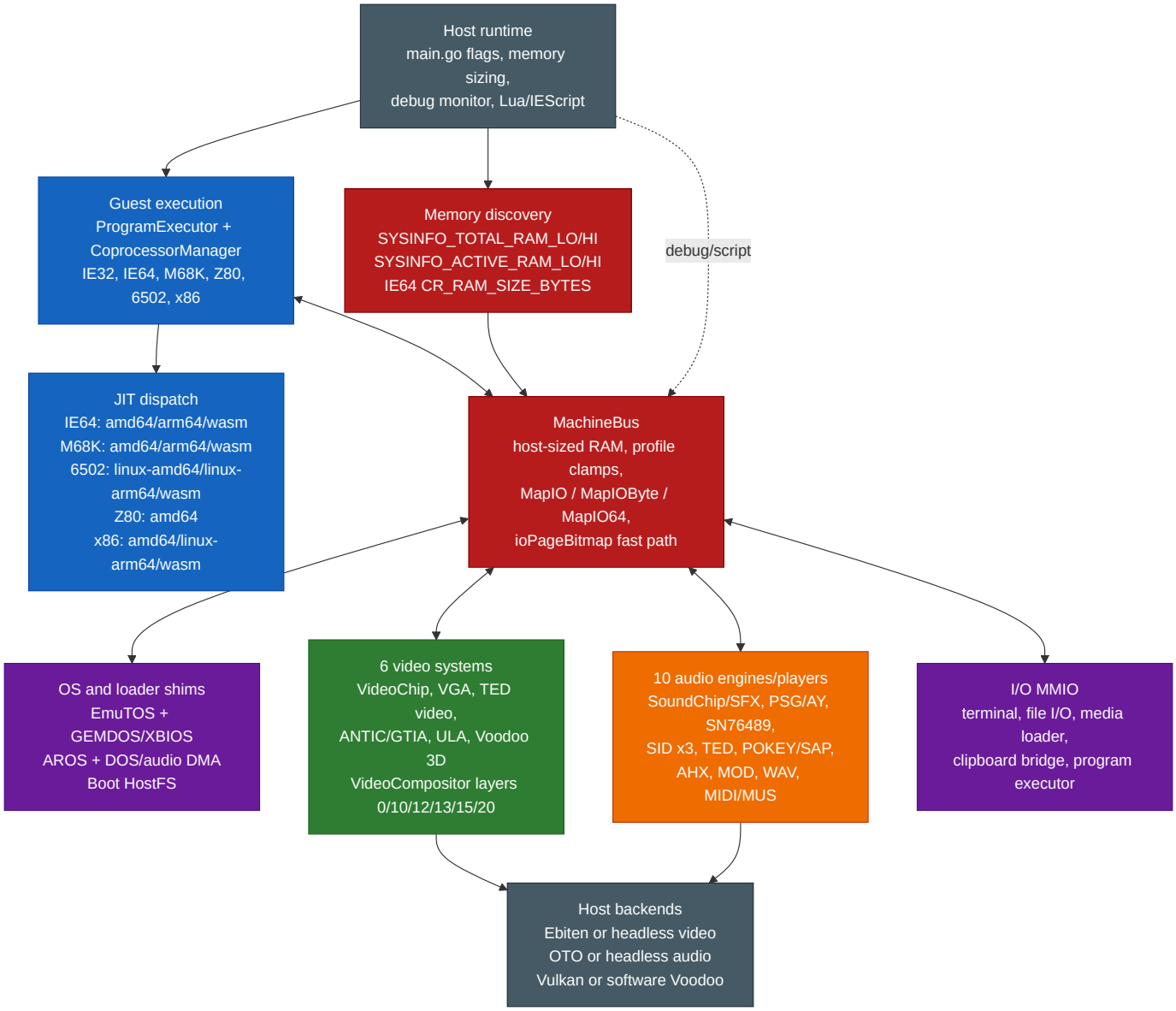
If a feature is platform-limited, the table or diagram names the supported platform path instead of implying that every implementation file is active in every build.

Single Complete Architecture Diagram

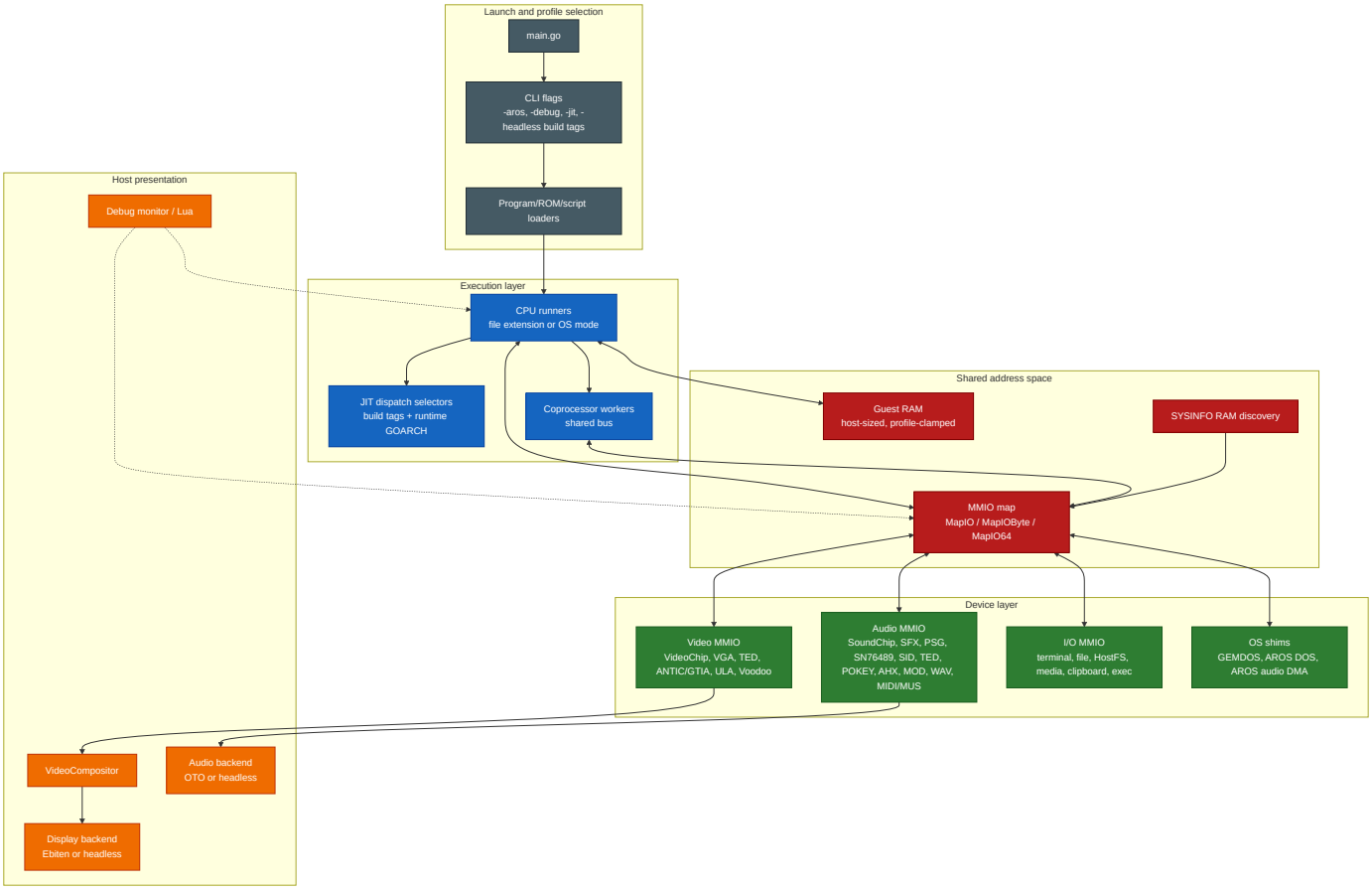

Audio systems
AY player
SID player
SAP player
TED player
AHX player
PSG / AY-3-8910
SN76489
SID1
SID2
SID3
POKEY
TED audio
AHX engine
MOD player
WAV player
MIDI/MUS player
SoundChip
10 flex channels + mixer
SFX trigger channels
Host process and tooling
main.go
CLI flags and boot profile
memory_sizing.go
host-sized guest RAM
profile_bounds.go
profile visible-RAM clamps
Host SDK and tools
assemblers, converters, IE64 C compiler, linker, archive tools, headers
Debug monitor
breakpoints, watchpoints, CPU-local and whole-machine snapshots
ScriptEngine
Lua / IEScript
Shared address space
Guest RAM
host-sized, profile-clamped
SYSINFO MMIO
total/active RAM
VGA windows
0xA0000 graphics
0xB8000 text
Video RAM
0x100000-0x5FFFFF
ULA VRAM aperture
0xFA000-0xFBAFF
Voodoo texture memory
0xD0000-0xDFFFF
AROS memory
Fast RAM + video RAM
Coprocessor memory slices
IE32, IE64, M68K, Z80, 6502, x86
Coprocessor mailboxes
0x790000-0x792FFF
Media staging buffer
0x800000-0x80FFFF
MachineBus
RAM, MapIO, MapIOByte, MapIO64, ioPageBitmap
Guest execution
ProgramExecutor
EXEC MMIO and CPU switching
CoprocessorManager
worker lifecycle and tickets
Debug CPU adapters
IE32, IE64, M68K, Z80, 6502, x86
IE32 interpreter
IE64 interpreter
M68K 68020 interpreter
6502 interpreter
Z80 interpreter
x86 interpreter / amd64, Linux arm64 and wasm JIT paths
IE64 JIT
amd64 + arm64
6502 JIT
Linux amd64 + arm64 + wasm
M68K JIT
amd64 + arm64 + wasm
Z80 JIT
amd64 only
x86 JIT
amd64 + Linux arm64 + wasm
OS and loader shims
EmuTOS loader
AROS loader
Bootstrap HostFS
File I/O MMIO
Media loader
GEMDOS intercept
XBIOS intercept
AROS DOS handler
AROS Paula-style audio DMA



1. Whole-System Architecture


Host runtime
main.go flags, memory sizing,
debug monitor, Lua/IEScript
Guest execution
ProgramExecutor +
CoprocessorManager
IE32, IE64, M68K, Z80,
6502, x86
Memory discovery
SYSINFO_TOTAL_RAM_LO/HI
SYSINFO_ACTIVE_RAM_LO/HI
IE64 CR_RAM_SIZE_BYTES
debug/script
JIT dispatch
IE64: amd64/arm64/wasm
M68K: amd64/arm64/wasm
6502: linux-amd64/linux-arm64/wasm
Z80: amd64
x86: amd64/linux-arm64/wasm
MachineBus
host-sized RAM, profile clamps,
MapIO / MapIOByte / MapIO64,
ioPageBitmap fast path
OS and loader shims
EmuTOS + GEMDOS/XBIOS
AROS + DOS/audio DMA
Boot HostFS
6 video systems
VideoChip, VGA, TED video,
ANTIC/GTIA, ULA, Voodoo 3D
VideoCompositor layers
0/10/12/13/15/20
10 audio engines/players
SoundChip/SFX, PSG/AY, SN76489,
SID x3, TED, POKEY/SAP,
AHX, MOD, WAV, MIDI/MUS
I/O MMIO
terminal, file I/O, media loader,
clipboard bridge, program executor
Host backends
Ebiten or headless video
OTO or headless audio
Vulkan or software Voodoo

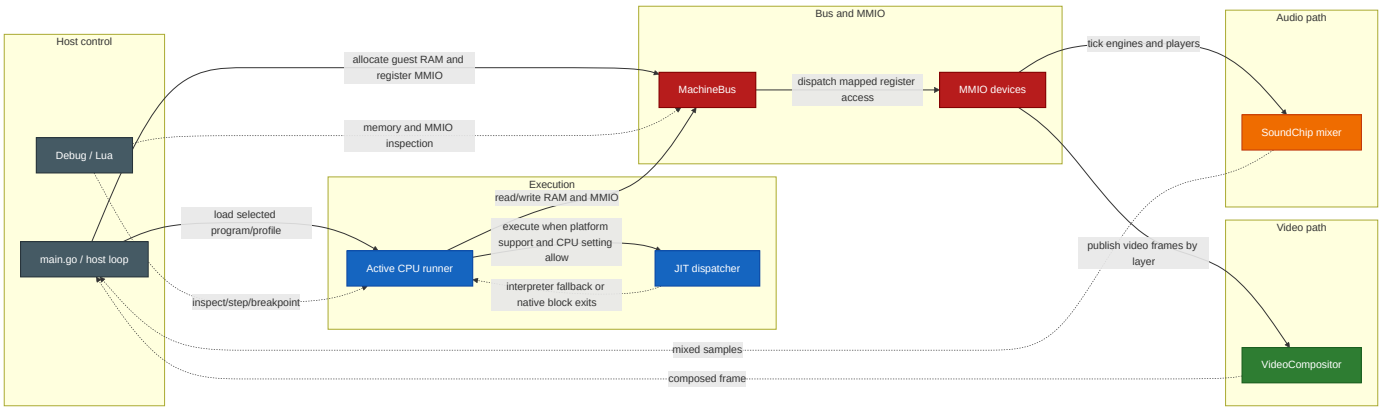
2. Layered System Overview



Bus architecture notes:

- **Concurrent multi-CPU bus** - the Program Executor selects the primary CPU mode, but the Coprocessor Manager can launch additional worker CPUs that run concurrently on the same bus. Address dispatch is immutable after mapping is sealed, `ioPageBitmap` provides the fast path, and I/O callbacks protect their own mutable state. There is no central bus-arbitration model exposed to guest software.
- **No centralised interrupt controller** - each CPU has per-CPU interrupt lines (IRQ/NMI as `atomic.Bool`). Peripherals signal the active CPU directly.
- **MMIO dispatch** - the bus uses an `ioPageBitmap []bool` fast path (page = 256 bytes). Non-I/O pages use direct unsafe pointer access with zero dispatch overhead.

Runtime Data and Control Flow



Subsystem Matrix

Subsystem	Runtime surface	Primary files	Wired registration / dispatch
CPU cores	IE32, IE64, M68K, Z80, 6502, x86	`cpu_*.go`, `cpu_*_runner.go`	`main.go` selects runners by file extension, OS mode, or EXEC MMIO
JIT	IE64 and M68K on amd64, arm64 and wasm; 6502 on Linux amd64 and arm64 plus js/wasm; Z80 on amd64; x86 on amd64, Linux arm64 and wasm	`jit_dispatch.go`, `jit_6502_dispatch.go`, `jit_6502_dispatch_wasm.go`, `jit_m68k_dispatch*.go`, `jit_z80_dispatch.go`, `jit_x86_dispatch*.go`	Build tags plus `runtime.GOARCH`; unsupported hosts use dispatch stubs. Linux arm64 and js/wasm 6502 lower the documented NMOS set in eligible direct RAM and resume through the interpreter at observation boundaries; js/wasm x86 uses the WebAssembly driver cache and declines activation entirely when SIMD support is absent
Bus and RAM	Host-sized guest RAM, profile clamps, MMIO, byte/64-bit handlers	`machine_bus.go`, `memory_sizing.go`, `profile_bounds.go`, `sysinfo_mmio.go`	`main.go` registers devices before execution; `MachineBus.SealMappings` prevents late maps
Machine lifecycle	Load resolution, reset quiesce, CPU/profile recreation, monitor/runtime rewiring	`machine_lifecycle.go`, `main.go`	`main.go` owns concrete devices; `Machine` applies reset/load orchestration through injected dependencies and profile targets
Video	VideoChip, VGA, TED video, ANTIC/GTIA, ULA, Voodoo	`video_chip.go`, `video_vga.go`, `video_ted.go`, `video_antic.go`, `video_ula.go`, `video_voodoo.go`	`main.go` maps each register/VRAM block and registers compositor layers 0/10/12/13/15/20
Audio	SoundChip/SFX, PSG/AY, SN76489, SID x3, TED, POKEY/SAP, AHX, MOD, WAV, MIDI/MUS	`audio_chip.go`, `sfx_trigger.go`, `psg_engine.go`, `sn76489_chip.go`, `sid_engine.go`, `ted_engine.go`, `pokey_engine.go`, `ahx_player.go`, `mod_player.go`, `wav_player.go`, `midi_player.go`	`main.go` maps chip/player MMIO and registers sample tickers/mixers into SoundChip; register-mapped players share `PlayerControlState` for staged playback requests
OS integration	EmuTOS, AROS, GEMDOS/XBIOS, M68K DOS bridge, Paula-style DMA	`emutos_loader.go`, `aros_loader.go`, `gemdos_intercept.go`, `aros_dos_intercept.go`, `aros_audio_dma.go`	OS profiles install their intercept MMIO and loader state during boot/reset; flat M68K mode installs the host-backed DOS bridge directly

Subsystem	Runtime surface	Primary files	Wired registration / dispatch
Tooling	Assemblers, disassembler, converters, IE64 C compiler, linker, archive tools, generators, and public headers	`assembler/`, `cmd/ie32to64/`, `cmd/ie64-cproc/`, `cmd/ie64ld/`, `cmd/ie64-ar/`, `cmd/ie64-ranlib/`, `internal/ie64meta/`, `internal/ie64obj/`, `internal/ie64link/`, `internal/ie64archive/`, `sdk/include/intuitionengine.h`	Makefile builds individual SDK tools and the Linux x86-64 Host SDK. The Host SDK combines first-party tools, IE64 C runtime and libraries, public assembly includes, the target-selected C hardware header, and user documentation

Internal Ownership Notes

- **IE64 opcode metadata** - `internal/ie64meta/table.go` is the source of truth for generated IE64 opcode constants and mnemonic/name tables used by the runtime CPU, monitor disassembler, standalone assembler, standalone disassembler, and internal assembler package. Build commands for `ie64asm` and `ie64dis` build `./assembler` with the relevant tag so generated files are compiled with the tagged tool entry point.
- **Machine lifecycle** - `Machine` in `machine_lifecycle.go` owns reset/load sequencing through dependency fields and profile targets. It preserves the existing guest-visible reset contract while keeping quiesce, failed-load rollback, CPU recreation, profile loading, and monitor/runtime rewiring out of the host event loop. A hard reset restages the configured coprocessor service after coprocessor reset and before CPU restart, so the service name pointer and worker-start path remain available across reset. Resetting or reloading flat M68K mode closes the current DOS bridge handles and locks, removes its MMIO mapping, clears runtime ownership, and installs one fresh bridge for the reloaded program.
- **Audio playback control** - register-mapped music players use `PlayerControlState` for staged pointer/length registers, optional high pointer, bus reads, busy/error/loop state, subsong selection, and async generation invalidation. This covers SID, PSG/SN/SNDH, TED, POKEY/SAP, AHX, MOD, WAV, and MIDI/MUS without changing their public MMIO contracts.
- **Video scheduling** - `VideoScheduler` centralises ticker ownership for the compositor and migrated video render-loop shims. Tests can drive it manually; migrated engine files do not own `time.NewTicker` directly. A measurement pass weighed folding these clocks into one deadline-driven timing service and found nothing to gain: an idle machine wakes exactly once per frame per clock, about 120 times a second for the compositor and one active video source, with no surplus polling, and the CPU cores that would spin while a guest is halted already park on a backoff sleep rather than a busy loop. Idle CPU sits at a fraction of a percent of one core, all of it in the Go scheduler's own park path. The `IE_PROFILE=1` harness in `timing_idle_profile_test.go` records those numbers, so a future timing service is a measured decision rather than an assumed one.
- **Voodoo software wrappers** - headless and `novulkan` builds share `softwareVoodooBackend` forwarding. The tagged Voodoo files keep feature registration and constructor selection only, so their `VoodooBackend` method sets stay identical.
- **IE64 MMU walk sharing** - CPU translation and bootstrap hostfs guest-pointer translation share the page-table walk result (`MMUSharedWalkResult`). CPU policy applies trap causes and A/D updates; hostfs policy requires `PTE_U` and leaves A/D bits untouched.
- **Performance accounting** - `IE_PERF_ACCT=1` enables per-CPU JIT/interpreter time, instruction, subsystem, and deopt counters; disabled accounting avoids atomic hot-path updates. Deopt reasons are `unsupported`, `helper`, `mmio`, `smc`, `interrupt`, `cache_pressure`, and `debug`. These counters are diagnostics only and do not change guest-visible execution. When `IE_PERF_ACCT` is enabled, the subsystem report is written to standard error once during clean shutdown or terminal-signal shutdown; `IE_PERF_ACCT_OUT` also writes it to a file. This includes a no-activity report when no instrumented counter advanced.

- **Boot collection point** - IE64 BASIC startup and a full reset that reloads BASIC force one Go collection after image loading and before CPU, compositor, render-loop, and audio startup. This collects transient loader allocations without imposing a recurring runtime collection policy.
- **IE64 JIT tiers** - amd64 Linux, Windows and macOS builds, plus Linux arm64, promote hot static control-flow chains into bounded multi-block regions. Regions retain selected GPR and FPU state across internal edges, preserve budget and retired-count accounting on back edges, and spill architectural state at every helper or dispatcher exit. MMU region promotion is enabled by default and can be disabled with `IE64_JIT_REGION_MMU=0`. The browser backend emits the same bounded region shape as one structured wasm function and keeps exact disjoint code ranges for SMC invalidation. MMU-off low-memory entries with an external conditional or register-indirect jump can instead record the executed path one block at a time. Accepted paths reuse the region compiler. Rejected paths rebuild any valid static fallback after checking the invalidation generation. Conditional predecessors are truncated after the hot branch and retain a cold side exit to the discarded fall-through tail.
- **IE64 loop specialisations** - invariant-address memory prechecks and bounded immediate-counter budget elision were implemented and tested for native blocks, native regions and structured wasm loops. Their conservative matchers rejected MMU, stack, changing-pointer, helper-capable and alternate-entry cases, selected only one loop per compiled unit, and retained store-side SMC probes. Ten Linux amd64 samples improved the targeted medians by 0.3 per cent and 1.1 per cent respectively. Both positive results are accepted. Failed speculative prechecks use internal fallback value 2 and interpret one instruction without recording an MMIO deoptimisation.

Platform JIT Matrix

The host-side JIT support is intentionally asymmetric and follows the dispatch files, not emitter-file presence. Release amd64 builds target x86-64-v3 (`GOAMD64=v3`, the Makefile default) for the best Go-compiler codegen (AVX2, BMI1/2, FMA), but this is a recommendation, not a hard requirement: the JIT only *requires* SSE4.1, which it emits unconditionally for IE64 FINT (ROUNDSS). The amd64 backend detects this at runtime via CPUID; on a host without SSE4.1 `initJIT` (via `checkJITHostFeatures`) declines to enable the JIT and the CPU falls back to the interpreter rather than aborting, so plain `go run .` / `go build` at the default `GOAMD64=v1` works on any x86-64 CPU (older hosts run interpreted). AVX2/BMI2/LZCNT paths are separately runtime-gated via `x86Host` and are never emitted unless the host supports them.

Host platform	JIT-enabled guest cores	Dispatch files
Linux amd64	IE64, 6502, M68K, Z80, x86	`jit_dispatch.go`, amd64 per-core dispatch files
Linux arm64	IE64, M68K, 6502, x86	`jit_dispatch.go`, `jit_m68k_dispatch_arm64.go`, `jit_6502_dispatch.go`, `jit_x86_dispatch_arm64.go`; 6502 lowers the documented NMOS set in eligible direct RAM and resumes through the interpreter at observation boundaries
Windows amd64	IE64, M68K, Z80, x86	amd64 per-core dispatch files
Windows arm64	IE64, M68K	IE64 and M68K dispatch; other non-IE64 cores use stubs
macOS amd64	IE64, M68K, Z80, x86	amd64 per-core dispatch files
macOS arm64	IE64, M68K	IE64 and M68K dispatch plus Darwin arm64 JIT write-protect helpers
Browser (js/wasm)	IE64, M68K, 6502 and x86 (wasm bytecode backends)	`jit_exec_wasm.go`, `jit_wasm_runtime.go`, `jit_m68k_dispatch_wasm.go`, `jit_6502_dispatch_wasm.go`, `jit_x86_dispatch_wasm.go`

Every available JIT backend starts enabled for directly constructed CPUs, normal runners, Program Executor launches, and JIT-capable coprocessor workers. `--nojit` selects interpreter execution for the primary CPU started from the command line. An unavailable backend always falls back to the interpreter; stopped primary CPUs can also be switched through IEScript.

The 6502 JIT is available only on Linux amd64, Linux arm64, and js/wasm; Windows and macOS use the interpreter. Native and wasm backends invalidate cached code after physical RAM writes and resume unsupported or observed accesses through the interpreter.

On macOS amd64, the JIT reuses the shared x86-64 host backends. On macOS arm64, executable memory uses the native `MAP_JIT` model with thread-pinned write protection toggles. IE64 and M68K have arm64 backends; the other guest cores remain interpreter-only.

On js/wasm the native backends are absent because the browser gives Go no executable memory. IE64, M68K, 6502 and x86 use separate wasm bytecode backends instead; see the backend sections below. IE32 and Z80 interpret in the browser.

M68020 JIT Backends

M68020 JIT backends are available on amd64 and arm64 Linux, Windows and macOS, plus js/wasm; the wasm backend requires `__goMem` and `M68K_WASM_JIT=0` disables it. Native runners enable the backend through the normal M68K JIT setting. Unsupported, cold or guarded instructions return to the interpreter without changing the M68020 architectural contract.

The M68020 JIT shares an untagged scanner, admission rules, CCR liveness, region formation and tier policy while keeping native and wasm lowering target-specific. M68020 JIT memory guards use the profile-visible RAM ceiling, and native and wasm stores invalidate compiled code before stale execution. The wasm backend additionally checks the captured guest bytes before entry and uses a code-page bitmap to stop self-modifying structured loops.

Hot M68020 native blocks can form bounded regions using constant-address propagation, constant-only folding, loop-invariant hoisting, observed-path cold exits and safe JSR leaf fusion. Indirect branch and return caches accelerate repeated targets, while chain and loop budgets force periodic dispatcher returns for interrupt and exception checks. The wasm backend uses structured in-function control flow rather than native executable memory and preserves the same retired-count, fallback and profile-boundary rules.

IE64 wasm JIT backend (js/wasm)

The browser build cannot emit machine code, but it can emit WebAssembly and let the browser's engine compile it. The wasm backend keeps the native JIT's shape and reuses its shared front end:

- **Scanner and context.** Blocks come from the same `scanBlock` and operate on the same `JITContext` through the `jitCtxOff*` field offsets. GOARCH=wasm is a 64-bit Go target (8-byte `uintptr`), so the layout is identical; a layout guard test (`jit_wasm_ctx_layout_test.go`) pins this on both native and js runs.
- **Encoder and translator.** `jit_wasm_encoder.go` writes wasm binaries; `jit_wasm_ie64_emit.go` translates a scanned block into one wasm function with signature `(ctxPtr i32) -> ()`. Both are untagged pure Go, so the differential test suite (`jit_wasm_ie64_diff_test.go`) executes generated blocks under wazero on native hosts and compares registers, PC, retired counts and guest RAM against the interpreter, instruction class by instruction class. Interpreter semantics are the reference wherever the amd64 emitter differs (Q-size ALU immediates zero-extend).
- **Shared memory.** Generated modules import the machine's own linear memory (`env.mem`; the hosting page exposes `instance.exports.mem` as `__goMem`), so blocks read and write the register file, guest RAM and the context in place at the same addresses the native backends use.
- **Exit protocol.** Blocks write `RetPC`/`RetCount` and return. MMIO accesses, out-of-window addresses and stack faults exit through the same `HELPER_*` protocol as the native backends; the Go-side dispatcher (`jit_wasm_runtime.go`)

services the helper via the interpreter's memory and stack primitives and resumes. Generated code never calls back into Go.

- **Instruction coverage.** An explicit allowlist (`wasmSupportedOpcode`) delimits the backend: the MMU-off integer core (data movement, ALU, load/store, conditional branches, BRA/JMP/HALT, JSR/RTS/PUSH/POP/JSR_IND), FP32 loads, stores and basic arithmetic, and FP64 loads, stores, arithmetic, transforms, comparisons and conversions. The IE64 wasm JIT executes FP32 FMOD and transcendental operations, and FP64 DMOD and transcendental operations, through helper exits that apply the processor FPU operation and resume the compiled block. Direct floating-point paths reproduce FPSR condition codes and sticky exception flags. A hot block is compiled up to its longest supported prefix; other instructions stay on the interpreter.
- **Regions and residency.** Hot static BRA chains compile as one bounded wasm function. Forward edges and one structured back edge retain hot GPRs in `i64` locals and FP64 pairs in `f64` locals. Every external, helper, SMC and budget exit commits the corresponding architectural state and dynamic retired count. Exact disjoint member ranges drive code-page indexing and SMC invalidation; same-page regions with data gaps fall back conservatively.
- **MMU gate.** While `cpu.mmuEnabled` is true the runtime neither enqueues compiles nor enters installed blocks; both halves of the gate are tested under node.
- **Self-modifying code.** Generated STOREs (including PUSH/JSR return address writes) probe the code-page bitmap exactly like the native emitters; a dirty store commits, publishes its exact address range and ends the block. The runtime resolves the range against a page-to-blocks index and drops only the blocks whose compiled bytes were actually written. A probe hit that overlaps no block is a false share (data sharing a 256-byte page with compiled code, which the EhBASIC image contains) and drops nothing; treating false shares as full flushes put RUN AOT into a permanent recompile storm.
- **Tiering.** The dispatcher (`jit_exec_wasm.go`) interprets through `StepOne` and counts visits to block-start PCs; at the hot threshold a compile is submitted through `WebAssembly.instantiate`, which is asynchronous. Promises resolve during the cooperative yield, so block installation costs the machine nothing. The backend is on by default; `IE64_WASM_JIT=0` (mapped from `/demo/?jit=0`) disables it, and it also stays off when the hosting page does not expose `__goMem`.

Dispatch between compiled blocks stays inside wasm: a driver module (`wasmBuildDriverModule`) reads the next PC from the context, looks it up in a direct-mapped PC-to-slot cache in linear memory and `call_indirect`s through a shared funcref table until the chain budget runs out, the lookup misses, or a block reports a helper exit or an SMC invalidation. The measured basis (node, V8): roughly 15 ns per in-wasm indirect call against roughly 700 ns per JS-boundary `Invoke`, which is why per-block Go round trips are reserved for helper exits. On the node hot-loop benchmark (`TestWasmJIT_Node_EndToEndParity`, 2 million iterations) the chained JIT runs 5.3 times faster than the interpreter.

IE64 JIT Shared Analysis Order

Each IE64 compilation unit runs its shared analyses in a fixed order before emission: FPSR condition-code liveness, loop analysis (including fixpoint hoist selection of loop-invariant chains for single-block pure integer loops), constant-only folding, then FP residency planning. Loop hoisting is decided before folding, and suppressed hoisted instructions are never replaced by folded constants. Folding treats plain `STORE` as constant-preserving and `LOAD` as invalidating only its destination; FP, control-flow and system instructions remain full barriers. Region compiles run the same order with folding applied per block; region plans never hoist.

Native observed regions outline every adjacent-forward conditional cold exit after the emitted hot path. Each outlined exit retains its own retired-count base, architectural spill mask, and pending FPSR condition codes. Backward hot edges keep an inline exit; the wasm backend keeps cold exits inside structured conditional arms instead of using native out-of-line stubs.

IE64 JIT 64-bit Execution Model

The IE64 JIT (amd64 and arm64) executes code, accesses data, and operates the stack at any `uint64` virtual or physical address, matching the interpreter. There is no `uint32` truncation on the PC, data addresses, stack pointer, branch targets, or chain targets. The one stack exception is the amd64 non-MMU fused-leaf path (documented below): it raw-indexes `[MemBase+SP]` and so assumes the SP is in the low window.

`SEI64` and `CLI64` are emitted as per-instruction interpreter bails (not NOPs) so they mutate `interruptEnabled` under the JIT. External device interrupts are delivered through a record-only sink and a pending mask polled at instruction and block boundaries by both the interpreter and the JIT dispatcher; see the "External Interrupt Delivery" section of this manual for the full model.

- **Low memory window**: each guest sees a contiguous low RAM window backed by the dense `cpu.memory[]` slice. All full-machine modes, including the IE64 family, size the window at the full 32-bit range (`busMemMaxBytes`) so a flat 32-bit program launched from the BASIC REPL can occupy the same address space it would get from a direct boot; on mmap-backed hosts the window is reserved lazily and resident memory grows only with pages actually touched, while `busMemBootClamp` keeps non-mmap hosts at a heap-safe size. The actual `len(bus.memory)` is `min(autodetected TotalGuestRAM, busMemCap)` (main.go), so on a small host the window can be smaller than the cap. Addresses above the window resolve through the bus's 64-bit physical path, backed by the `Backing` interface; the split derives from `len(bus.memory)`, so the two regions are complementary by construction. IE64 BASIC keeps its whole interpreter arena (programme text, variables, strings, stack) inside the dense 32-bit window so every pointer stays 32-bit clean; the arena simply grows to fill that window, up to just below the resident stack. The SYSINFO low-window register pair (0xF2414/0xF2418) reports the dense window size so the stack, heap top and AOT arena are all bounded by it rather than by the larger backed total. Production IE64 boots on Linux/darwin bind high RAM via `MmapBacking`; `SparseBacking` is the test implementation. SparseBacking uses a sparse two-level atomic page table: missing pages read as zero, writes allocate leaves and pages with compare-and-swap, same-page word operations read or write the page slice directly, and reset publishes a fresh empty table. Hot RAM accesses load each I/O bitmap snapshot once per word or long access. After mappings are sealed, the bus reuses a quiescence-published snapshot pointer on the hot path; debug access also has a quiescence-updated plain active mirror with the service atomic retained as a backstop. M68K JIT invalidation is called through the bus's cached invalidator when one is registered. High physical addresses are supported for code fetch and for data / stack *operands* (the latter via the helper exit, including a high SP), with one caveat below.
- **High-PC stack-op caveat**: a block *fetched from* high physical backing that itself contains a stack op (`PUSH`/`POP`/`JSR`/`RTS`/`JSR_IND`) is not JIT-compiled -- `ExecuteJIT` takes the `highPhys && containsStackOp(instrs)` path and runs the block one instruction at a time through `interpretOne()` (which routes the stack through the bus). This is pinned by `TestJIT_HighPC_StackOpBlock_BailsToInterpreter`. Stack ops in blocks fetched from the low window are JIT-compiled and, for a high SP, route through the helper exit -- **except** the fused-leaf path below.
- **Fused-leaf high-SP exception**: on amd64 with the MMU off, `scanBlock` may fuse a `JSR64` to a small leaf subroutine (`ie64FusedJSRLeafCall` / `ie64FusedRTSLeafReturn`, gated by `ie64ScanJSRLeafFusionEnabled` -- amd64 only). The fused emit (`jit_emit_amd64.go`) handles the call/return push/pop as **raw** `[MemBase+SP]` traffic *before* the normal `emitJSR_AMD64`/`emitRTS_AMD64` high-SP guards, so it does **not** take the helper exit. This is safe only because the marker is suppressed whenever `mmuBail` is set, but `mmuBail` for fused leaves is set by `compileBlockMMU` (MMU mode) only -- in non-MMU mode a fused leaf with an SP outside the low window would read/write past `cpu.memory[]`. Non-MMU guests are therefore expected to keep the stack in the low window; high-SP correctness via the helper exit holds for unfused stack ops, not for fused leaves.
- **Direct fast path vs. helper exit**: native code uses a direct `[memBase+addr]` access only when the MMU is off **and** the address is in the low window (size-aware bound: `addr <= MemSize - accessBytes`). Otherwise it takes the JITContext-mediated *helper exit*.

- **Helper exit protocol**: native code cannot safely call back into Go (it runs on g0 via `asmcgocall`), so on a high/MMU access it writes a structured request to the `JITContext` (`NeedHelper`, `HelperAddr`, `HelperSize`, `HelperVal`, `HelperRd`, `HelperPC`, `LiveSP`), flushes the live SP and the faulting instruction's PC, and returns through the epilogue. `ExecuteJIT` dispatches the request (`handleJITHelper`) and re-enters the JIT. Helper ops cover `LOAD`/`STORE`, `FLOAD`/`FSTORE`, `DLOAD`/`DSTORE`, `PUSH`/`POP`, and the control-flow `JSR`/`RTS`/`JSR_IND`.
- **Helper resume**: when a helper retires exactly one instruction and the CPU state still matches the emitted continuation, `ExecuteJIT` may resume directly at that native continuation. IE64 helper resume is enabled by default and can be disabled with `IE64_JIT_RESUME=0`, `false`, `off`, or `no`. IE64 helper resume is cancelled by timer delivery, debug breakpoints, pending invalidation, PC changes, MMU mode changes, or PTBR changes.
- **Interpreter parity**: helpers use `cpu.loadMem`/`storeMem` for data and `cpu.mmuStackWrite`/`mmuStackRead` for the stack -- the same paths as the interpreter. JSR sets PC to the call target, RTS sets PC to the popped return address, and JSR_IND sets PC to `rs + sext(imm32)` -- the displacement is sign-extended, so a negative `imm32` (bit 31 set) subtracts (with `rs == R31` based on the decremented SP). The FP load/store side effects differ: FLOAD and DLOAD set the destination FP register **and** the FP condition codes; FSTORE and DSTORE read the FP register and write memory, leaving FP registers and condition codes unchanged -- matching the interpreter in each case.
- **MMU rule**: when the MMU is enabled, non-atomic data/stack ops take the helper exit (virtual-to-physical mapping is not a direct offset). The dispatcher refreshes `ctx.MMUEnabled` before every `callNative` so the native guards are never stale.
- **JIT MMU micro-TLB**: IE64 JIT MMU helpers use a four-entry read/write micro-TLB for translated low-window RAM pages. AMD64 and ARM64 probe it for `LOAD` and `STORE`; ARM64 store hits retain physical self-modifying-code invalidation. Helper dispatch fills the cache only after a successful translation to dense RAM below `IO_REGION_START`; high physical backing and MMIO are not inserted. Native probes include the access type and CPU privilege/SUA/SKAC/SKEF mode bits in the key. `TLBFLUSH` clears the IE64 JIT micro-TLB and `TLBINVAL` invalidates matching micro-TLB VPN entries. Dispatch refreshes the mode prefixes before every native call.
- **Fault contract**: the handler sets `cpu.PC = HelperPC` before the operation, so `trapFault` records the correct `faultPC`; faulting instructions are not counted and re-execute after the trap. Pre-decrement ops (PUSH/JSR/JSR_IND) roll the SP back on a trapping fault.
- **Atomics carve-out**: atomics (CAS/XCHG/FAA/FAND/FOR/FXOR) do not use the helper-exit protocol. The native emitters run sequentially-consistent host-atomic fast paths only for aligned, non-MMU, low-window RAM addresses; MMU-on, high-address, MMIO, or unaligned cases bail to the interpreter so the canonical `atomicRMW64` trap and bus semantics apply. `compileBlockMMU` also sets `mmuBail` on fused JSR/RTS leaf markers, so under MMU the `OP_JSR64`/ `OP_RTS64` emit takes the `mmuBail` branch to `emitBailToInterpreter` (a whole-instruction interpreter bail with its own fault/accounting path) -- neither the raw fused fast path nor the guarded helper exit runs. In non-MMU mode `mmuBail` is unset, so the raw fused fast path is used (see the fused-leaf high-SP exception above).
- **Range-scoped self-modifying code**: IE64 self-modifying-code tracking uses 256-byte guest code pages and physical code-page tracking for MMU-compiled blocks. The 256-byte page mark is a prefilter: helper-side writes and native-store dispatcher exits with a known nonzero virtual range validate that range against `JITBlockCoveredRanges` before mutating the cache. Non-MMU checks scan only the non-MMU cache; MMU checks are scoped to the current `PTBR`, so a same-VA block in another address space does not cancel helper resume, record an SMC deopt, increment invalidation stats, or evict cached code. When a compiled store overlaps cached code, the dispatcher invalidates only the changed guest range; MMU stores that overlap compiled physical backing through an alias still fall back to whole-cache invalidation. x86 self-modifying-code tracking uses 256-byte code pages and range invalidation.

x86 JIT backends

The x86 guest core has a native amd64 JIT on Linux, Windows and macOS, a Linux/ARM64 direct-prefix backend, and a js/wasm backend gated on wasm SIMD. The guest target is the 32-bit flat-model i386 base plus the intentionally supported `BSWAP` and an x87 FPU, lowered through SSE2 on amd64 where eligible; there is no 486/Pentium/MMX/SSE guest requirement. Blocks are scanned and compiled with the fixed Tier 1 register mapping plus per-block Tier 2 frequency allocation, native EFLAGS passthrough, block chaining, and self-loop and experimental multi-block region formation.

Linux/ARM64 keeps guest registers memory-resident and directly lowers a verified register subset plus guarded MOV, ALU, TEST, 16-bit and 32-bit IMUL, Group 3 NOT/NEG, LES/LDS, count-one SHL/SHR/SAR and immediate SHLD/SHRD memory forms, including 16-bit immediate counts through 16. Guard failure returns to the interpreter before mutation; a native write to a compiled 256-byte page publishes an exact invalidation range. Its x87 forms use direct lowering for reset, status, sign-only forms and finite register `FADD ST(0),ST(i)`; the remaining supported forms use the canonical decoded helper exit. The finite valid-tag register `FADD`, `FMUL`, `FSUB`, `FSUBR`, `FDIV` and `FDIVR` forms are directly lowered; every tag transition and exceptional result resumes through that helper.

Linux arm64 x86 executes a verified direct subset and resumes remaining forms through the interpreter.

The js/wasm backend publishes the manifest-backed direct and canonical x87 helper families through imported guest memory, a WebAssembly table driver, native chaining, loop regions and conditional region promotion. Public availability is all-or-nothing: when wasm SIMD support is missing the complete x86 wasm JIT stays unavailable and x86 interpretation continues normally. The x86 wasm JIT requires its executable coverage manifest, WebAssembly SIMD, the Go memory export, and `X86_WASM_JIT` not equal to `0`; otherwise x86 continues in the interpreter.

Memory accesses at runtime-computed addresses go through a page-safety and I/O bitmap check on the fast path. Native stack and word emitters (`PUSH`/`POP` r32, near `CALL`/plain `RET`, memory-source `MOVSX Gv,Ew`) validate their whole access span against the guest-visible RAM ceiling (`ProfileMemoryCap()`, not the backing slice length) before any load, store or `ESP` change, and bail to the interpreter on cross-page, out-of-bounds or I/O spans. Instructions with far control-flow, interrupt, port-I/O or complex-flag semantics stay on the interpreter. Unsupported x87 forms, including segmented and address-size memory forms, use a CPU-local decoded helper snapshot so the canonical interpreter handler does not re-read mutable guest instruction bytes. Dynamic amd64 x87 exits use the same context ABI, which carries the instruction bytes, PC, CS, decoded fields, resolved effective address, access width and segment before control returns to the interpreter.

Runtime switches are `X86_JIT_CHAINS` (default on), `X86_JIT_REGIONS` (default off), `X86_JIT_RTS` (default off), `X86_JIT_STATS` (profiling, default off), the browser-only `X86_WASM_JIT` switch, shared `IE_PERF_ACCT` accounting, and the shared `IE_SIMD` kill switch. Region formation is experimental and must be enabled explicitly. Unsupported or guard-failing instructions leave through the normal interpreter fallback, so these switches change execution strategy rather than guest-visible x86 semantics.

Build Profiles and Observable Runtime

Build tags change host backends, not the guest-visible ISA contract. The main guest bus, CPU cores, MMIO register addresses, assemblers, and script binding names remain the reference surface unless a specific backend is absent from the build. On amd64, release profiles target x86-64-v3 for codegen quality; lower `GOAMD64` levels still build and run, with SSE4.1 as the enforced runtime floor.

Profile	Build tags / knobs	Runtime effect visible to users
Default VM	no special tag	Ebiten display, Oto audio, Vulkan-backed Voodoo where available, and normal host integrations.
`novulkan`	`-tags novulkan`	Voodoo uses the software backend and does not require the Vulkan SDK. Guest Voodoo registers remain mapped.

Profile	Build tags / knobs	Runtime effect visible to users
`headless`	`-tags headless`	Display, audio backend, overlay, clipboard, and GUI integrations use stubs suitable for CI. CPU, bus, MMIO, scripting, and most device state paths still compile for tests.
`headless-novulkan`	`CGO_ENABLED=0 -tags "novulkan headless"`	Pure-Go portable VM build with headless stubs and software Voodoo path.
Browser (`make wasm`)	`GOOS=js GOARCH=wasm -tags embed_basic`	IE64, M68K, 6502, and x86 use WebAssembly JIT backends. The 6502 backend lowers documented NMOS instructions in eligible direct RAM and uses interpreter resume at mapping and observation boundaries. `IE64_WASM_JIT=0`, `M68K_WASM_JIT=0`, `P65_WASM_JIT=0`, and `X86_WASM_JIT=0` disable the corresponding browser backend; x86 also requires WebAssembly SIMD. IE32 and Z80 interpret. Ebiten renders to a WebGL canvas, Oto uses WebAudio, Vulkan is excluded, and guest RAM is a fixed 256 MiB heap backing. FileIO and Bootstrap HostFS use an in-memory volume seeded from web assets, with file contents fetched lazily on first read. CPU execution yields cooperatively so browser events, asynchronous compilation, video, and audio continue on the single WebAssembly thread.

Headless stubs should be treated as backend substitutes, not as a different machine model. A test can still write video or audio MMIO and inspect guest state, but there is no host window, host audio device, or GUI overlay to observe the result directly.

The x64 live image gives Oto an unreachable PulseAudio server so it selects ALSA, and `pipewire-alsa` carries that stream into PipeWire. The live-image AppArmor profile grants the standard audio abstraction and read access to the PipeWire configuration required by this route. This is a live-image host backend choice; it does not change the guest audio devices or their MMIO contracts.

The Linux x86-64 Host SDK packages `ie32asm`, `ie32to64`, `ie64asm`, `ie64dis`, `ie64-cproc`, `ie64ld`, `ie64-ar`, `ie64-ranlib`, QBE, cproc-qbe, the IE64 runtime and libraries, public assembly includes, `intuitionengine.h`, and user documentation. The x64 live image stages that archive and its SHA-256 file under `SDK/Toolchains`; the former per-platform `SDK/Tools` tree and standalone IE64 toolchain archive are not staged.

Browser builds use an IE64 WebAssembly bytecode JIT for supported MMU-off integer, FP32, and FP64 blocks, with interpreter fallback and `IE64_WASM_JIT=0` as the runtime disable switch. Browser FileIO and Bootstrap HostFS use in-memory volumes seeded from web assets, with file contents fetched lazily on first read.

The browser exposes an in-tab bridge over the same FileIO memory volume. `ieImportFile` adds a session-local file with a 64 MiB per-file limit, `ieExportFile` returns a saved file's bytes, and `ieDeleteFile` removes a file; none of these operations uploads data to a server. `ieTypeText` injects representable text bytes and `ieKey` injects the supported editing and navigation key sequences through the normal terminal input path. Imported files disappear when the page reloads.

Build Flags Outside Make

The Makefile targets export `GOEXPERIMENT=simd` and select `GOAMD64=v3` for the release profile, and they pass `-trimpath` and `-pgo=default.pgo`. A bare `go build .` run outside Make inherits none of that: it loses the v3 codegen level, compiles the scalar kernels instead of the amd64 SIMD ones unless `go env -w GOEXPERIMENT=simd` has been set on the machine, and produces a binary with untrimmed source paths. Profile-guided optimisation is the exception, because the toolchain picks up `default.pgo` from the main package directory automatically. Use the Make targets for anything whose performance is being measured.

CPU Profile Capture

`IE_CPUPROFILE=<path>` starts a CPU profile at boot and writes it when the machine shuts down cleanly, when a script calls the quit or exit binding, or when the process is interrupted from the terminal. The variable is unset by default and

profiling is entirely disabled when it is empty, so ordinary runs carry no profiling cost and the default signal disposition is unchanged.

Because `os.Exit` skips deferred cleanup, every exit path in `main.go` calls `exitProfiled` instead, which flushes an in-progress profile first and is equivalent to `os.Exit` when no profile is running. The interrupt handler re-raises the signal so termination behaves as it normally would, and exits explicitly if the re-raise is unavailable, which is the case on Windows.

The profiles produced this way are the input to `default.pgo`. To regenerate it, run `make pgo-regenerate` (which builds a `-pgo=off` capture binary, runs each manifest workload for its stated duration with `IE_CPUPROFILE`, merges the results and verifies the merged profile builds without fallback, writing `default.pgo.new` rather than overwriting in place); or do the same steps by hand. Confirm that `go build -pgo=default.pgo .` completes without a warning or a fallback. Record the revision, toolchain, workloads and durations in `default.pgo.manifest` beside the profile, and accept the new profile only when benchstat shows no regression against `-pgo=off` across the video, audio and bus benchmarks.

The Makefile passes PGO profiles explicitly: `PGO_PROFILE` selects native profiles and `WASM_PGO` selects wasm profiles; `make pgo-regenerate` writes `default.pgo.new`. This avoids relying on Go's root auto-discovery, so a regenerated root profile cannot silently change a target that should use its own. The amd64 targets (native, novulkan, headless, headless-novulkan, x86-64-v3) use `PGO_PROFILE` (default `default.pgo`). The wasm targets use `WASM_PGO`, which also defaults to `default.pgo`: a PGO profile is symbolic (function names and edge weights) and Go applies it in the arch-independent inliner, so `default.pgo`'s samples for the functions shared between the native and wasm builds (bus, IE64 interpreter, BASIC runtime, video kernels) drive the wasm codegen as well; entries for amd64-only functions such as the native JIT are dropped. The current profile adds approximately 0.2 per cent to the wasm binary size; performance acceptance still requires measurement against `-pgo=off`, because PGO is not an architectural guarantee against regression. Go cannot sample-profile js/wasm itself, because the single-threaded WebAssembly runtime has no SIGPROF-equivalent, so an `IE_CPUPROFILE` capture on wasm yields zero samples; a profile that also covers the wasm JIT paths must come from a browser or Node CPU capture converted to pprof, at which point `WASM_PGO=default.wasm.pgo` selects it. Each is a Makefile variable, so any target can be pointed at a target-specific profile.

Programme Benchmark Inventory

Each subsystem the performance programme touched has a named benchmark pair, run through the Makefile `bench-baseline`, `bench-after` and `bench-compare` targets and compared with benchstat. The inventory below is the reference set; a drift test asserts every benchmark named here still exists in the sources, so a rename cannot quietly drop a gate.

Subsystem	Benchmarks
Compositor	`BenchmarkComposite_StaticScene`, `BenchmarkComposite_SkipUnchanged`, `BenchmarkComposite_FullDirty`, `BenchmarkCompositeScaled_Resample`, `BenchmarkCompositeScanline_MultiLayer`
Software Voodoo	`BenchmarkVoodooRasterTextured`, `BenchmarkVoodooRasterScene`, `BenchmarkVoodoo_FullscreenQuad_Software`
Audio	`BenchmarkReadSamplesBlock_AllChips`, `BenchmarkAudioRegisterWrite_UnderRender`
Bus and memory	`BenchmarkRead32_NonIO`, `BenchmarkWrite32_NonIO`, `BenchmarkBusWrite32_RAM`, `BenchmarkBusWriteSpan_RAM`, `BenchmarkSparseBackingReadSpan`, `BenchmarkSparseBackingWriteSpan`, `BenchmarkDirtyMark`
Monitor	`BenchmarkMonitorHooks_Disabled`
IEScript	`BenchmarkIEScriptHotLoop`

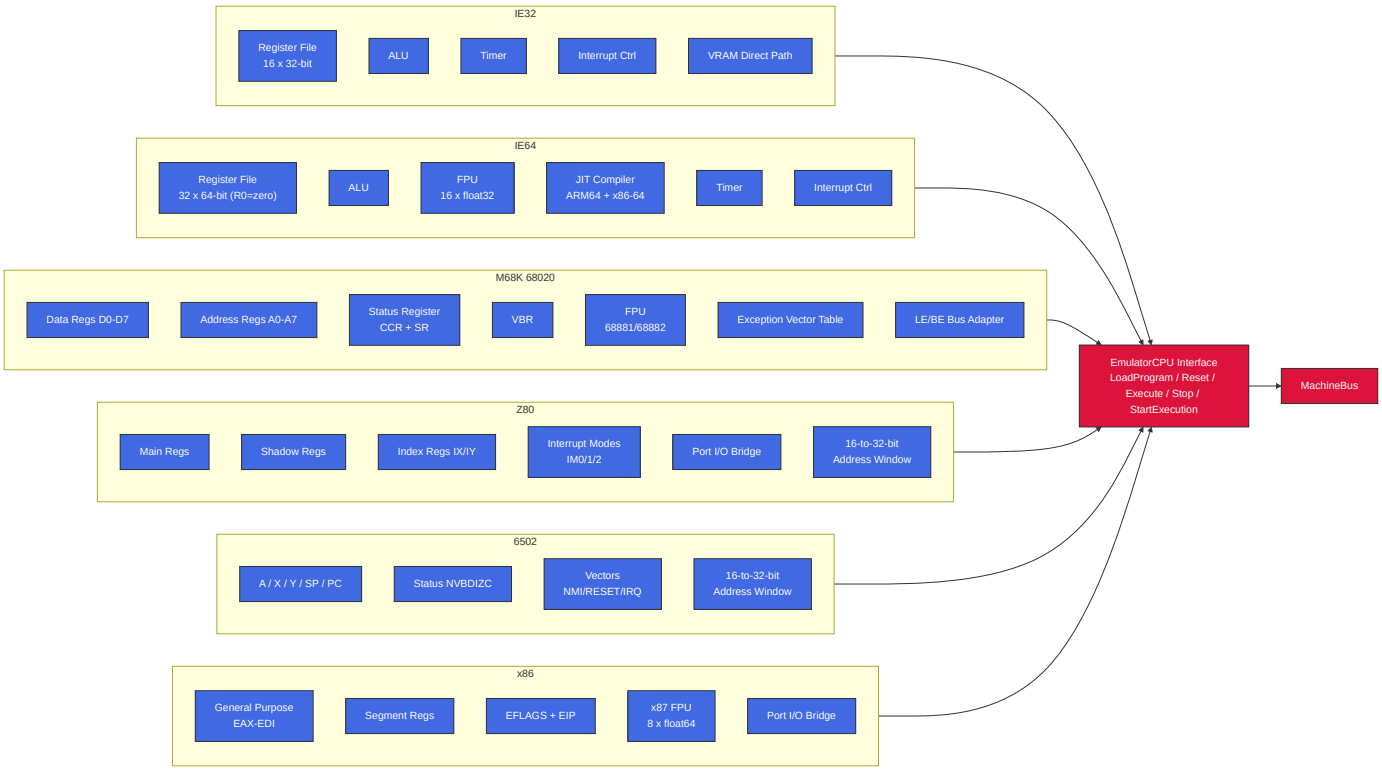
The `IE_BENCH_GATE` environment variable arms the mains-power benchmark uniformity gate; it is a measurement gate rather than a runtime switch and does not change any guest-visible behaviour.

IE64 BASIC Native Compilation

The resident IE64 BASIC environment implements `RUN AOT`, `COMPILE`, `TRANSPILE`, and `ASSEMBLE` with a typed linear-IR compiler resident in guest memory. The pipeline parses tokenised BASIC, validates the arena or standalone target, applies typed optimisation passes, lowers to IE64 operations, emits assembly, and invokes the in-guest assembler. Compiler workspace and generated output use the active-RAM AOT arena rather than a fixed host-side buffer.

The replacement compiler does not delegate unsupported statements to the removed legacy transpiler. Its source-consumed coverage inventory classifies each parser surface and records whether it is common to both targets, arena-only, standalone-runtime-backed, direct-only, or invalid. Target-specific unsupported constructs fail with compiler diagnostics; generated standalone images remain self-contained and do not depend on a resident BASIC interpreter.

3. CPU Subsystem



CPU Timers (IE32 / IE64)

Both IE32 and IE64 use CPU-internal countdown timers backed by atomic fields on the CPU structs.

- IE32 increments an internal cycle counter once per decoded instruction step, after fetch and operand resolution and before the decoded instruction body executes; when that counter reaches `SAMPLE_RATE`, it resets the counter and decrements the timer count by one.
- IE64 decrements the timer count once per decoded instruction step, after fetch/decode and before the decoded instruction body executes. When the IE64 timer is armed, JIT dispatch uses the CPU single-instruction path so an interrupt can be delivered before the body of the instruction whose decoded step expires the timer.
- On expiry, it sets timer state to expired and can trigger a CPU interrupt.
- If still enabled after interrupt handling, the count auto-reloads from the configured period.

There is no stable bus/MMIO timer control ABI at present. Legacy include symbols such as `TIMER_CTRL/TIMER_COUNT/TIMER_RELOAD` are retained for compatibility but are deprecated.

CPU Selection by File Extension

Extension	CPU	Address Space	Notes
`.iex` / `.ie32`	IE32	32-bit flat (clamped to active visible RAM)	Native RISC, 8-byte fixed instructions
`.ie64`	IE64	64-bit (sees full active visible RAM)	64-bit RISC, R0=zero, JIT on ARM64 + x86-64
`.ie68`	M68K	32-bit flat (clamped to active visible RAM)	Bare 68020, big-endian with LE bus adapter
`.ie65`	6502	16-bit + bank windows	Bank windows reach the banked-CPU visible ceiling
`.ie80`	Z80	16-bit + bank windows + port I/O bridge	Bank windows reach the banked-CPU visible ceiling
`.ie86`	x86	32-bit flat (clamped to active visible RAM) + compatibility bank windows	Ordinary addressing is flat; bank/VRAM windows are compatibility overlays
`.tos` / `.img`	M68K	EmuTOS profile (`EmuTOS_PROFILE_TOP`)	EmuTOS boot with GEMDOS intercept
`.ies`	Script	N/A	Lua scripting engine (IE Script)

Bare `.ie68` uses the active-visible RAM ceiling; EmuTOS and AROS M68K loader modes use profile bounds.

AROS boot is selected by CLI mode (`-aros`, optional `-aros-image`), not file extension.

Z80 Port I/O Bridge

Z80 `IN`/`OUT` instructions are translated to bus MMIO accesses by the `Z80BusAdapter`:

Z80 Port	Chip	Bus Target	Protocol
`$A0-$AD`	VGA	`0xF1000`	Direct register map (MODE, STATUS, CTRL, SEQ, CRTC, GC, DAC, DAC read index, DAC mask, VRAM bank)
`$B0-$B7`	Voodoo 3D	`0xF8000`	Addr lo/hi + 4 data bytes (write to `$B5` triggers 32-bit bus write)
`$D0/$D1`	POKEY	`0xF0D00`	Register select / data
`$D4/$D5`	ANTIC	`0xF2100`	Register select / data (x4 stride)
`$D6/$D7`	GTIA	`0xF2140`	Register select / data (x4 stride, collision regs through `0xF21F8`)
`$E0/$E1`	SID	`0xF0E00/0xF0E30/0xF0E50`	Register select / data; select bits 5-6 choose SID1/SID2/SID3
`$E4/$E5`	SN76489	`0xF0C30/0xF0C31`	Data write / last-written read and ready-status read
`$F0/$F1`	PSG	`0xF0C00`	Register select / data
`$F2/$F3`	TED	`0xF0F00` / `0xF0F20-0xF0F6B`	Register select / data (audio indices `$00-$05`, video indices `$20-$32` x4 stride)
`$FE/$FD/$BE/$FA/$FB/$FC`	ULA	`0xF2000-0xF2014`	Border, control, status, VRAM address latch low/high, and paged VRAM data

Z80 16-bit Memory Translation

Z80 memory accesses in `0xF000-0xFFFF` are translated to bus `0xF0000-0xF0FFF`. ANTIC/GTIA access is provided through the Z80 port bridge (`$D4-$D7`) rather than general memory-mapped addressing.

x86 Port I/O Bridge

x86 shares the Z80 Voodoo port mapping (`$B0-$B7`) and most sound-chip port mappings, but **POKEY uses direct port-to-register mapping** (not the Z80's select/data protocol). x86 does not implement standard PC VGA I/O ports; VGA access is through the shared bus MMIO aperture and the direct `$A0000-$AFFFF` VRAM memory window.

x86 Port	Chip	Bus Target	Protocol
`$B0-$B7`	Voodoo 3D	`0xF8000`	Addr lo/hi + 4 data bytes
`$60-$69`	POKEY	`0xF0D00+(port-0x60)`	**Direct** - port offset maps to writable register
`$D4/$D5`	ANTIC	`0xF2100`	Register select / data (x4 stride)
`$D6/$D7`	GTIA	`0xF2140`	Register select / data (x4 stride, collision regs through `0xF21F8`)
`$E0/$E1`	SID	`0xF0E00/0xF0E30/0xF0E50`	Register select / data; select bits 5-6 choose SID1/SID2/SID3
`$F0/$F1`	PSG	`0xF0C00`	Register select / data
`$F2/$F3`	TED	`0xF0F00` / `0xF0F20-0xF0F6B`	Register select / data (audio indices `$00-$05`, video indices `$20-$32` x4 stride)
`$FE`	ULA	`0xF2000`	Border colour only (bits 0-2)

Key difference from Z80: x86 POKEY access is direct - ports `$60-$69` map one-to-one onto writable POKEY registers at `0xF0D00+(port-0x60)`. ANTIC (`$D4/$D5`) and GTIA (`$D6/$D7`) keep their own select/data pairs.

x86 also directly accesses VGA VRAM at `$A0000-$AFFFF` in the memory path (no port translation needed).

Bank Windows (Z80 / 6502 / x86)

The banked 6502/Z80 ABI uses bank windows to access a 32 MiB banked-CPU visible ceiling from a 16-bit address space; bank translation rejects addresses at or above that ceiling. x86 ordinary addressing remains flat 32-bit, but the x86 bus adapter also implements the same compatibility bank windows plus a VRAM bank window. Those x86 window translations are capped at the legacy `DEFAULT_MEMORY_SIZE` ceiling; ordinary x86 addressing remains clamped by the active visible RAM exposed through the shared bus.

CPU Address	Size	Purpose	Bank Select Register
`$2000-$3FFF`	8KB	Bank 1 (sprite data)	`$F700/$F701` (lo/hi)
`$4000-$5FFF`	8KB	Bank 2 (font data)	`$F702/$F703` (lo/hi)
`$6000-$7FFF`	8KB	Bank 3 (general)	`$F704/$F705` (lo/hi)
`$8000-$BFFF`	16KB	VRAM window	`$F7F0` (bank number)
`$F000-$FFF9`	4090B	I/O window, excluding 6502 vectors	Hardwired: `$Fxxx` -> bus `$F0xxx`
`$F200-$F24F`	80B	Coprocessor gateway subrange	Hardwired: -> bus `$F2340+offset`
`$FFFA-$FFFF`	6B	6502 vectors (NMI/RESET/IRQ)	Identity mapped

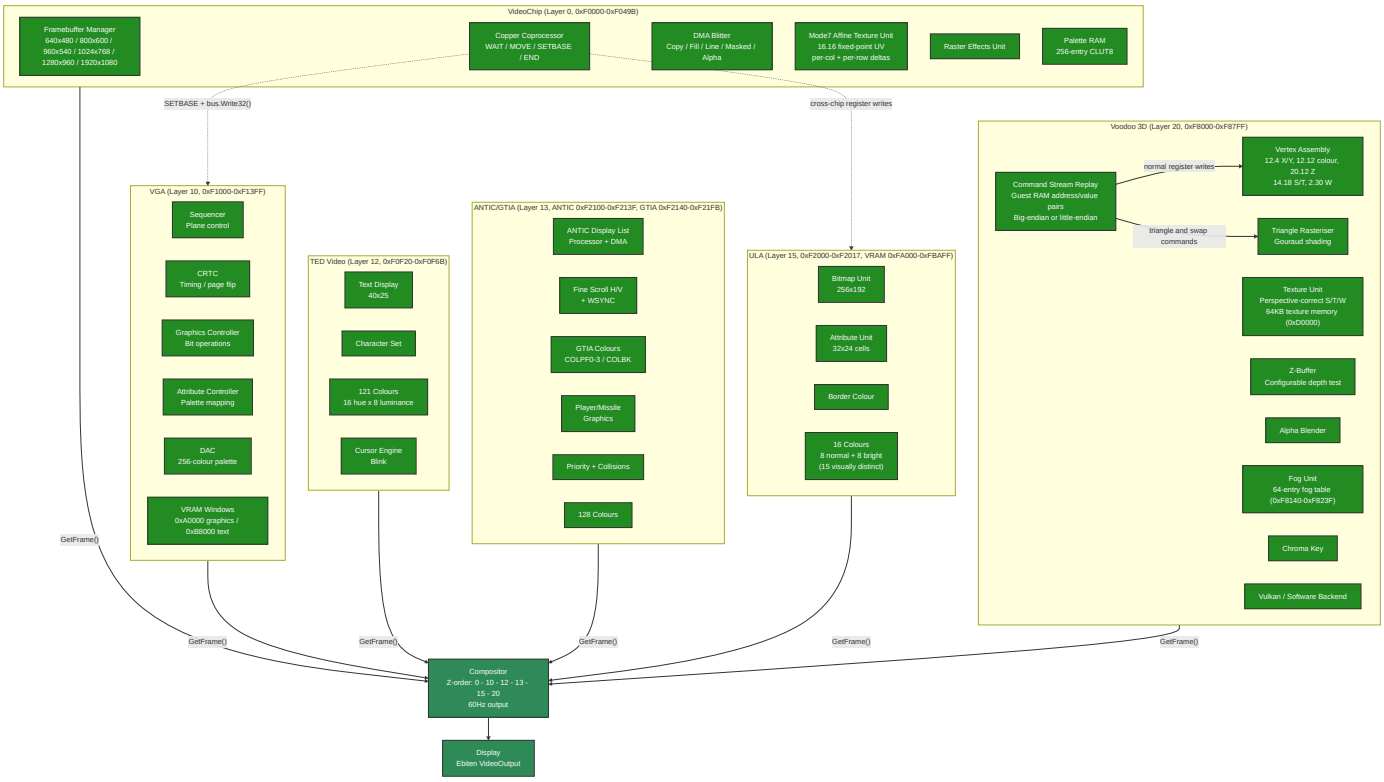
6502 I/O Chip Page Dispatch

The 6502 uses `ioTable[page]` to route memory-mapped I/O through the bus:

6502 Address	Chip	Bus Target	Notes
`$D200-$D209`	POKEY	`0xF0D00+offset`	
`$D400-$D40F`	PSG	`0xF0C00+offset`	
`$D500-$D51F`	SID1	`0xF0E00+offset`	32-byte window
`$D520-$D53F`	SID2	`0xF0E30+offset`	32-byte window
`$D540-$D55F`	SID3	`0xF0E50+offset`	32-byte window
`$D600-$D605`	TED Audio	`0xF0F00+offset`	
`$D620-$D632`	TED Video	`0xF0F20+offset x4`	Stride-4 register mapping including raster compare registers
`$D700-$D70D`	VGA	`0xF1000`	Direct handler call plus DAC read index, DAC mask, and VRAM bank
`$D800-$D817`	ULA	`0xF2000+offset`	Registers plus paged VRAM data port

ANTIC/GTIA intentionally has no 6502 `$D400/$D000` compatibility surface; `$D400-$D40F` is PSG on the 6502 map.

4. Video Subsystem



VideoChip routes 32-bit VRAM writes before the register switch, so ordinary framebuffer stores avoid the control-register decode path. Rectangle-shaped blitter fills in raster-op modes update the front buffer directly and mark the covered dirty tiles once when the destination stride is the display row stride; non-rectangular strides keep the per-pixel path for exact dirty tracking. Raster band writes through bus memory aggregate backing invalidation per row.

Voodoo Command Stream Replay

The Voodoo command stream reduces guest MMIO traffic by replaying an array of absolute register address and value pairs from guest RAM. Software writes the array address to `VOODOO_CMD_PTR` (`0xF833C`), its pair count to

VOODOO_CMD_COUNT (0xF8340), then writes a replay value to VOODOO_CMD_SUBMIT (0xF8344). The engine reads the complete stream into a scratch buffer and applies it through the normal Voodoo register path under one lock. This retains the side effects and ordering of individual register writes without requiring a guest MMIO exit for every word.

VOODOO_CMD_SUBMIT_REPLAY (1) selects the original big-endian pair format. VOODOO_CMD_SUBMIT_REPLAY_LE (2) selects little-endian pairs. These values describe the byte order of the command buffer, not the CPU that submits it, so any CPU may use either format when its software constructs a matching buffer. If both bits are present, the little-endian selection takes precedence.

A stream may contain at most 65,536 pairs. Misaligned addresses, addresses outside the Voodoo register block, and recursive writes to the three command stream control registers are skipped.

Voodoo 3D State Binding

Voodoo triangle submission is state-machine driven. A write that commits VOODOO_TRIANGLE_CMD binds the current Voodoo raster state to that triangle: VOODOO_FBZ_MODE, VOODOO_ALPHA_MODE, VOODOO_FBZCOLOR_PATH, VOODOO_TEXTURE_MODE, fog state, chroma key, stipple, clip rectangle, slope registers, and the currently uploaded texture. Later register writes or texture uploads do not affect already submitted triangles. VOODOO_SWAP_BUFFER_CMD may flush the queued batch later, but the batch is rasterised in triangle submission order using each triangle's bound state.

Voodoo texture slots retain immutable uploaded textures by slot identifier; VOODOO_TEX_BIND selects a resident texture for subsequently submitted triangles without another guest-memory transfer, and SYSINFO advertises the slot contract. Write the slot identifier to VOODOO_TEX_SLOT before VOODOO_TEX_UPLOAD to retain that upload. Binding an empty or out-of-range slot leaves the current texture unchanged. The SYSINFO_FEATURE_VOODOO_TEX_SLOTS bit identifies machines that implement these registers.

When IE_SWAP_HASH=1, each presented VOODOO_SWAP_BUFFER_CMD receives a deterministic sequence number and a 32-bit FNV-1a frame hash captured immediately after readback. VOODOO_SWAP_HASH_SEQ (0xF8358) returns the latest sequence; write a sequence to VOODOO_SWAP_HASH_QUERY (0xF835C) and read VOODOO_SWAP_HASH_VALUE (0xF8360) to retrieve it. The 64-entry history returns zero for unknown or evicted sequences. Hashing is disabled by default, but swap sequence numbering still advances when readback produces no frame.

Consecutive triangles may share an internal state snapshot until a raster-state register or texture upload changes; that sharing is not guest-visible. Fog-table and palette raster lookups remain compatibility-pending. The software backend is the conformance reference; it rasterises each flush from per-flush copies of the bound snapshots under a framebuffer lock, so live raster-state register writes are never blocked by an in-flight raster. Vulkan renders multi-state frames natively by binding each snapshot's pipeline, scissor, push constants, and texture per state group inside one command buffer, and is the only render path in windowed builds: the software rasteriser is the headless build's renderer and the conformance reference, never a runtime fallback. Flushes that arrive without a guest clear render standalone under the clear pass by default; an experimental load-op render pass variant (IE_VOODOO_LOAD_PASS=1) composites them over the previous GPU frame. Pipeline or descriptor creation failures drop the frame and log the error rather than silently downgrading.

Voodoo swap jobs run asynchronously; oversized triangle batches render mid-frame without presentation or swap callbacks, and STATUS exposes busy and SWAPBUF while a presented swap is pending. VOODOO_SWAP_BUFFER_CMD hands the current batch to the swap worker, and VOODOO_STATUS reports framebuffer and SST busy plus SWAPBUF while that presented swap is active. A later swap waits only when the swap pipeline is full (two jobs in flight), giving up to two frames of run-ahead. If a frame exceeds VOODOO_MAX_BATCH_TRIANGLES, the full batch is rendered into the draw buffer as a mid-frame render-only flush, without presenting the frame or firing swap callbacks, and triangle submission continues into a fresh batch. This preserves oversized tiled frames instead of dropping triangles.

The native Vulkan backend defers the present fence wait to the next frame, enabled by default and disabled with IE_VOODOO_ASYNC_PRESENT=0. A present submission (frame N) is queued without blocking on its fence; the wait is

drained at the start of the next frame's record, by which point a whole guest CPU frame has elapsed and the GPU work is already complete, so the wait no longer stalls. Until then the readback output holds the previously completed frame, so consumers observe frame N-1 while the renderer produces frame N. Because the persistent staging map (above) already makes the readback copy itself effectively free, a single draw image is kept and the overlap comes from hiding the fence wait behind the guest CPU frame rather than from duplicating GPU images. Swap-hash and duplicate-frame tracing force the deferred readback to complete first (`FinishPendingReadback`) so they observe the exact frame swapped; with the switch off every present waits synchronously in `SwapBuffers`. The N-1/N contract is pinned by the native-Vulkan gate `TestVulkanAsyncPresent_NMinus1Contract`.

The native Vulkan backend folds the frame-final render and the framebuffer readback into one submission, enabled by default and disabled with `IE_VOODOO_ONE_SUBMIT=0`. The swap worker's presentation flush records the frame-final render pass without submitting it (`FlushTrianglesForPresent`), and `SwapBuffers` appends an explicit colour-to-transfer image-memory barrier and the image-to-buffer copy into the same command buffer, then issues a single `vkQueueSubmit` and one fence wait, removing the separate render submission and its fence wait. Mid-frame overflow flushes and direct `FlushTriangles` callers keep their immediate-submit semantics. An empty (clear-only) frame still records and holds a final command buffer so one submission always occurs. With the switch off the render submits on its own and `SwapBuffers` does its own readback submission as before. The two paths are proven to produce a bit-identical framebuffer by the native-Vulkan gates `TestVulkanOneSubmit_MatchesTwoSubmitEmptyFrame` and `TestVulkanOneSubmit_MatchesTwoSubmitTriangle`.

The native Vulkan backend keeps a persistent host mapping of its readback staging buffer, enabled by default and disabled with `IE_VOODOO_PERSIST_MAP=0`. The staging memory is host-coherent, so it is mapped once when the staging buffer is created and every framebuffer readback copies straight from that mapping instead of a per-frame `vkMapMemory`/`vkUnmapMemory` pair. The mapping is dropped at the single staging-buffer teardown point, which covers resize, destruction and failed-initialisation cleanup, and re-established when the buffer is recreated. With the switch off the readback maps and unmaps per frame as before. The behaviour is proven on a real device by the native-Vulkan gate `TestVulkanPersistentStagingMap_ReadbackAndLifecycle` (headless builds wrap the software backend and never map device memory).

Copper Cross-Chip Bus Access

The copper coprocessor is internal to VideoChip but can write to any MMIO-mapped chip on the bus:

- **Default**: MOVE targets VideoChip's own registers (`copperIOBase = VIDEO_REG_BASE`)
- **SETBASE opcode** changes `copperIOBase` to any MMIO address (e.g. `VGA_BASE 0xF1000`)
- Subsequent MOVE instructions compute `regAddr = copperIOBase + (regIndex * 4)` and route via `bus.Write32()` to VGA, ULA, or any other chip's MMIO
- This enables per-scanline palette changes, mode switches, and register manipulation on any video chip
- `copperIOBase` resets to `VIDEO_REG_BASE` at the start of each frame
- The compositor's `ScanlineAware` interface orchestrates this: `StartFrame()` -> `ProcessScanline(y)` -> `FinishFrame()`

Extended Blitter: BPP Modes, Draw Modes, Colour Expansion, and Scale

The blitter supports two pixel formats via `BLT_FLAGS` (`0xF0488`): RGBA32 (4 bpp, default) and CLUT8 (1 bpp). Bits 4-7 select one of 16 raster draw modes (Clear, And, Copy, Xor, Invert, etc.) applied per pixel during FILL and COPY operations. `BLT_OP=4` performs source-over alpha blending with source alpha in bits 31-24 using `out = (src*a + dst*(255-a))/255`. With `BLT_FLAGS` bit 12 (`ALPHA_TMPL`) set, `BLT_OP=4` instead treats `BLT_SRC` as an 8 bpp alpha plane (row stride in `BLT_SRC_STRIDE`) and blends the `BLT_FG` colour over the destination with the same source-over equation. This serves antialiased glyph and icon rendering (the AROS driver uses it for `PutAlphaTemplate`). `BLT_OP=7` performs nearest-

neighbour scaling in RGBA32 or CLUT8. When BLT_FLAGS=0, the blitter defaults to Copy mode with RGBA32 for full backward compatibility.

Masked copies (BLT_OP=3) copy source pixels only where a 1-bit mask is set. The mask base address is in BLT_MASK, the mask row stride in BLT_MASK_MOD (0 selects packed rows of (width+7)/8 bytes), and BLT_MASK_SRCX gives the starting bit offset within each mask row. Mask bits are sampled LSB-first by default; setting BLT_FLAGS bit 11 (MASK_MSB) selects MSB-first sampling, which matches the Amiga PLANEPTR convention used by AROS layer masks, so the driver can pass CopyBoxMasked masks to the hardware without conversion.

BLT_CTRL bit 0 starts the synchronous blit, bit 1 is read-only busy, and bit 2 enables a completion pulse on IntMaskBlitter. The BLT_CTRL start edge samples the register shadow from the shared bus image in little-endian register order; VideoChip big-endian mode affects guest-facing register and pixel access, not the internal bus-shadow hydration order. BLT_STATUS bit 0 is ERR, bit 1 is DONE, and bit 2 is sticky IRQ_PENDING (write 1 to clear). Invalid opcodes, out-of-range Mode7 samples, overflowed blitter bounds, and destination rectangles outside CPU-visible writable memory set ERR and do not silently fall back to COPY or wrap into another address range.

The colour expansion operation (BLT_OP=6) renders 1-bit glyph templates into coloured pixels for hardware-accelerated text. It reads a template from BLT_MASK, uses BLT_FG/BLT_BG (0xF048C/0xF0490) as foreground/background colours, and supports three modes: JAM2 (opaque - set bits write FG, clear bits write BG), JAM1 (transparent - only set bits write FG), and Invert (set bits XOR the destination). BLT_MASK_MOD (0xF0494) sets the template row stride and BLT_MASK_SRCX (0xF0498) provides sub-byte bit alignment for glyph fragments. Template bits are MSB-first (Amiga convention).

Line drawing (BLT_OP=2) supports an extended mode when BLT_FLAGS != 0: BLT_DST becomes the framebuffer base address, BLT_WIDTH holds the packed endpoint coordinates (y1<<16)|x1, and BLT_DST_STRIDE sets the row stride. This allows line drawing into arbitrary bitmaps (not just the active framebuffer) with BPP awareness and all 16 draw modes. When BLT_FLAGS=0, legacy behaviour is preserved (endpoint in BLT_DST, base at VRAM_START). In extended mode the blitter does not clip - callers must provide pre-clipped coordinates (the AROS driver uses Cohen-Sutherland clipping before calling the blitter).

Scale blits (BLT_OP=7) use BLT_SRC/BLT_DST as base addresses, BLT_WIDTH/BLT_HEIGHT as the source size in pixels, and BLT_COLOR as a packed destination size (height << 16 | width). BLT_SRC_STRIDE defaults to source width times bytes-per-pixel. BLT_DST_STRIDE defaults to the active VideoChip mode stride for VRAM destinations or destination width times bytes-per-pixel for ordinary memory. In non-direct VRAM mode, visible VRAM accesses use the active VideoChip front buffer and off-screen VRAM aperture accesses fall back to bus memory so back buffers can be presented through VIDEO_FB_BASE. Out-of-bounds source or destination rectangles, rectangles crossing the VRAM aperture end, zero source dimensions, and zero destination dimensions set BLT_STATUS.ERR.

Raster-band register draws (VIDEO_RASTER_Y, VIDEO_RASTER_HEIGHT, VIDEO_RASTER_COLOR, VIDEO_RASTER_CTRL) target the active framebuffer source. In RGBA32 mode, a nonzero VIDEO_FB_BASE writes the band into bus memory at that base; otherwise it writes the internal front buffer or direct VRAM slice. In CLUT8 mode, raster bands write indexed bytes at VIDEO_FB_BASE.

Register	Address	Description
BLT_FLAGS	0xF0488	BPP (bits 0-1), draw mode (bits 4-7), JAM1/invert flags (bits 8-10), MSB-first mask sampling (bit 11), alpha-template blend (bit 12)
BLT_FG	0xF048C	Foreground colour for colour expansion
BLT_BG	0xF0490	Background colour for colour expansion
BLT_MASK_MOD	0xF0494	Template row modulo (bytes per row)
BLT_MASK_SRCX	0xF0498	Starting X bit offset in template

Mode7 Affine Texture Unit

The blitter's Mode7 operation (`bltOpMode7`) implements SNES-style affine texture mapping as a DMA blitter mode within VideoChip. It uses 8 dedicated MMIO registers (`0xF0058-0xF0074`):

Register	Address	Description
`BLT_MODE7_U0`	`0xF0058`	Starting U coordinate (16.16 fixed-point)
`BLT_MODE7_V0`	`0xF005C`	Starting V coordinate (16.16 fixed-point)
`BLT_MODE7_DU_COL`	`0xF0060`	U increment per column (16.16)
`BLT_MODE7_DV_COL`	`0xF0064`	V increment per column (16.16)
`BLT_MODE7_DU_ROW`	`0xF0068`	U increment per row (16.16)
`BLT_MODE7_DV_ROW`	`0xF006C`	V increment per row (16.16)
`BLT_MODE7_TEX_W`	`0xF0070`	Texture width mask (must be power-of-2 minus 1)
`BLT_MODE7_TEX_H`	`0xF0074`	Texture height mask (must be power-of-2 minus 1)

The rasteriser walks each destination pixel, computes the source UV from the affine matrix (origin + column delta + row delta), wraps via power-of-2 bitmask, and samples the source texture. This enables rotation, scaling, and perspective-like effects on tiled backgrounds - the same technique used by the SNES PPU2 for its Mode 7 background layer.

VideoChip Mode7 honours the `BLT_FLAGS` BPP field: RGBA32 samples and writes 4-byte pixels, while CLUT8 samples and writes 1-byte palette indices with BPP-aware default strides. In RGBA32 mode the default source stride is `(texture mask width + 1) * 4`; in CLUT8 mode the default source stride is `(texture mask width + 1)`. The destination stride follows the same BPP-aware default-stride rules as the other blitter operations: active framebuffer width in bytes for VRAM destinations, or destination width times bytes-per-pixel for ordinary memory.

Video Compositor

The compositor collects immutable frame snapshots from all enabled video sources and blends them in Z-order (layer 0 at the back, layer 20 at the front). For IEVideoChip CLUT8 mode, mapped VRAM and direct bus-backed VRAM are converted through the palette before compositing. CLUT8 conversion is cached only when `fbBase` points at the VideoChip-visible VRAM window; direct RAM-backed framebuffer modes are converted every frame because guest CPU stores can bypass VideoChip dirty tracking. Desktop startup uses a 1920x1080 fullscreen presentation by default, and the x64 live-image launcher locks it fullscreen through `IE_LIVE_IMAGE=1`. Native sources keep their requested dimensions. The default native mode is 960x540 for exact 2x 1080p presentation. Video compositor default scale mode is stretch-fill; F11 toggles non-16:9 sources to aspect-fit. `Shift+F11` toggles fullscreen/windowed mode when that mode is not locked.

Two rendering paths:

1. **Scanline-aware path** - used when at least one enabled source implements `ScanlineAware`. The compositor advances scanline-capable sources in sorted layer order for each scanline, then blends all enabled sources in the global layer order. If exactly one source also implements `ScanlineBatchAware`, it advances the whole scanline range under one source lock. Multi-source scanline composition keeps per-scanline interleaving because that ordering is guest-visible.
2. **Full-frame fallback** - used when no enabled source is scanline-aware. It collects complete frames and blends them in sorted layer order. Same-size frame blending uses persistent strip workers instead of spawning goroutines each frame. Sources can optionally report dirty rectangles; outputs that implement `RegionUpdatingOutput` receive region uploads, while other outputs keep the full-frame upload path. Sources marked `OpaqueFrameSource` can use a promotion-aware copy path.

All-zero frame pixels are transparent; any nonzero alpha or RGB value is opaque. During compositing, zero-alpha nonzero-RGB pixels are promoted to opaque `0xFFRRGGBB` before they overwrite the destination. The compositor tick remains fixed at 60 Hz for guest VBlank compatibility; `GetRefreshRate()` reports the output backend rate, while `GetTickRate()` reports the compositor tick.

FrameGenerationSource lets the compositor skip collect/copy/blend/upload work only after source TickFrame hooks run and only when every enabled source generation is unchanged. A pending full-frame update, an active pixel consumer, an enabled source without generation tracking, or a scanline-compositing source disables this skip for that tick. The unchanged-frame composite skip is enabled by default and can be disabled with `IE_VIDEO_COMPOSITE_SKIP=0`; logical frame timing still advances on skipped ticks.

The frame callback is split into two responsibilities so the skip preserves the timing contract. An unconditional logical-frame/timing callback (script frame count, EmuTOS VBL delivery, `frameChan` wake) fires on every tick, including skipped ones, and advances the compositor frame counter and timestamp; a skipped tick therefore still advances the logical frame and delivers VBL without materialising pixels. A separate pixel-consumer callback (the recorder) fires only after a composed tick. While a pixel consumer is live (recording or capturing), a pixel-consumer-active predicate disables the skip so materialised pixels exist every tick. This split is pinned by `TestCompositeSkip_TimingFiresPixelsDoNot`, `TestCompositeSkip_KillSwitchForcesComposite` and `TestCompositeSkip_ActivePixelConsumerDisablesSkip`.

Video frame leases are enabled by default and can be disabled with `IE_VIDEO_FRAME_LEASES=0`. Copy-buffer paths use three-slot lease rings to keep source-layer buffers alive while the hardware compositor or snapshot path still references them. Sources that implement the compositor copy interface can copy a stable frame directly into the caller-provided lease buffer, avoiding an intermediate source snapshot before compositor handoff. The software output path can also retain a lease-backed final frame for `UpdateFrame` instead of copying through the legacy output buffer. Frame leases keep compositor handoff buffers stable until release; hardware layers retain leases or stage copies when leases are unavailable. Snapshot APIs still return deep copies.

Tile-based retained composition is enabled by default and can be disabled with `IE_VIDEO_TILE_COMPOSITE=0`. The software path composes into a lease ring slot, and a slot's pixels survive until that slot is handed out again, so the slot already holds a complete composite of an earlier frame. Composition repaints only the 64 by 64 tiles dirtied since that earlier frame and retains the rest untouched. Per-frame dirty tile sets are kept in a short history so the union across the intervening frames is exact. A frame whose change extent is unknown marks every tile: any layer that publishes no dirty rectangles, a scanline-composited frame, a forced full frame, or a layer set whose geometry, order or blend mode changed. Tiles own disjoint destination pixels and read only immutable layer buffers, so above a small tile count they are composed on parallel workers. Every tile is composed with the same per-destination-pixel rules as the full-frame path, so the output is bit-identical either way.

A layer whose source declares its frames opaque skips alpha normalisation entirely. Every consumer of such a layer forces alpha itself: the software kernels OR `0xFF000000` into each pixel they copy, on the whole-frame, scaled and tiled paths alike, and the hardware compositor's shader does the same, so the frame that reaches the display is identical either way. Normalising anyway costs a full pass over every frame, which a browser profile of the wasm build showed as its largest single function. Sources that do not declare themselves opaque are still normalised, because the blend path relies on it.

Regional uploads are enabled by default and can be disabled with `IE_VIDEO_PARTIAL_UPLOAD=0`. The upload planner clips the compositor's dirty rectangles to the frame, drops empty and already-covered ones, and abandons regional upload in favour of one whole-frame upload once the regions cover more than 60 per cent of the frame or exceed sixteen in number. It packs region pixels into a scratch buffer it owns, so a steady state of the same regions uploads without allocating. A change of frame geometry discards the plan and the scratch, and sends the next frame whole, because nothing the backend retained still describes the frame.

Scaled composition samples each destination column through a resample plan built once per source and destination width pair and cached on the compositor. Where the destination is no narrower than the source, any eight consecutive destination

pixels read from a window of at most eight consecutive source pixels, so the plan also carries a vector form that serves those eight pixels with a single cross-lane permute. Narrowing scales, unaligned windows and the sub-block tail defer to the scalar gather, which is the canonical reference.

Tiled software Voodoo rasterisation is enabled by default and can be disabled with `IE_VOODOO_TILE_RASTER=0`. A triangle flush is binned once, in submission order, over a 64 by 64 tile grid, and workers then take whole tiles rather than row bands of a single triangle. A tile owns its colour and depth pixels exclusively and replays its own binned list in submission order, so depth-equal, blended, fogged, chroma-keyed and overlapping translucent primitives land exactly as they would serially and the framebuffer is bit-identical to the per-triangle path. Tiles are indexed in raster rows, before the Y-flip, which is a bijection on rows and so preserves the disjointness. `IE_VOODOO_WORKERS` overrides the worker count, with `1` replaying every tile on the calling goroutine. Flushes still complete before `FlushTriangles` returns, so `WaitSwapIdle` keeps meaning that all raster work has finished.

Pooled Voodoo texture uploads are enabled by default and can be disabled with `IE_VOODOO_TEXPOOL=0`. The guest texture bytes are read into a reusable pooled scratch buffer, then a single fused pass swaps big-endian to little-endian directly into both the retained per-upload buffer and the resident texture memory, removing the separate in-place swap and texture-memory copy passes. The returned upload buffer stays a fresh per-upload allocation because it is retained immutably by the raster-state snapshot and texture slots. With the switch off the legacy allocate-read-swap-copy path runs. The two paths are proven byte-identical (returned buffer and texture memory) by `TestVoodooTexPool_MatchesUnpooledByteForByte`.

GPU-side compatibility-format conversion is enabled by default and can be disabled with `IE_VIDEO_GPU_CONVERT=0`. It needs a shader-capable Ebiten backend, so headless builds convert on the CPU whatever the switch says, and the CPU converter remains the canonical oracle. CLUT8 conversion runs as a Kage fragment shader over one retained source texture, which holds the 256 palette entries in the rows above the packed index rows, four indices to an RGBA texel. Palette and indices share a texture because Ebiten requires every source image bound to a shader draw to be the same size, and the texture is only as wide as the index rows need, with the palette wrapping across that width, because Ebiten cannot write part of an image: every `WritePixels` replaces the whole texture. A 320 by 200 frame therefore uploads about 64 KiB, against 256 KiB for the expanded RGBA frame the CPU path uploads.

The shader is gated by a render and readback differential against the CPU expander, not by a pure-Go mirror alone, since only a real render exercises the compiled shader, the texture formats, the bindings and the coordinate rules. Ebiten needs the main OS thread for that, which a test function never has: creating a texture from one faults inside the driver, and reading pixels back outside a running game panics with "ReadPixels cannot be called before the game starts". Such checks therefore register a named body and re-execute the test binary with `IE_GPU_GATE_BODY` set to that name, and `TestMain` hands the main thread to Ebiten before any test state exists. The hardware compositor readback tests use the same mechanism, which is what they needed once Ebitengine moved to v2.10.

Retained hardware-compositor layer textures are enabled by default and can be disabled with `IE_VIDEO_RETAINED_LAYERS=0`. Each RGBA hardware layer keeps its uploaded texture between frames and skips the whole-image `WritePixels` when the retained texture already holds the same source's unchanged content, decided by a per-layer content generation: a source that tracks a frame generation reports it, and any other source gets a per-frame unique value so its layer always re-uploads (the conservative fallback, which never claims unchanged pixels it cannot prove). The quad vertices and the uniform/option storage are cached per layer and rebuilt only when the source or destination geometry changes. With the switch off, every layer uploads and rebuilds its draw state each frame as before. The skip and geometry-reuse decisions are pinned by `TestEbitenRetainedLayer_UploadSkipDecision` and `TestEbitenRetainedLayer_GeomReuseDecision`, and the end-to-end upload skip by the GPU-gated `TestEbitenOutput_HardwareCompositor_RetainedLayerSkipsUpload`.

The live path carries the frame as indices end to end. A source in CLUT8 mode publishes `IndexedFrameForCompositor`, the compositor hands the layer on unexpanded when the conversion is wanted and the output implements `AcceptsIndexedLayers`, and the Ebiten backend stages the indices and expands them with the shader as it draws. Every

other case keeps the RGBA path: the kill switch, a headless or software output, a source that is not in an indexed mode, and the software fallback taken when a hardware frame update fails. A backend that cannot run the conversion shader is handled at the backend, not by the compositor, which cannot rescue it because the frame update it accepted had already reported success: the failing frame is expanded on the CPU so it still presents correctly, and the failure latches so `AcceptsIndexedLayers` returns false and every later frame arrives as RGBA. Indexed layers are expanded on the CPU at one choke point in the software renderer, so nothing downstream needs to know a layer ever arrived as indices, and a retained snapshot layer takes its own copy of the indices because the compositor reuses that scratch each frame.

The planar formats (VGA, ULA, TED, ANTIC) are not implemented, since their per-scanline register state is not the fixed index arithmetic a fragment shader can mirror.

The audio event ring is enabled by default and can be disabled with `IE_AUDIO_EVENT_RING=0`, which restores the synchronous barrier path. It queues guest register writes so a CPU write does not wait for the renderer's segment mix, and applies them at the same boundary the synchronous path used. Section 5 describes the admission handshake and the barrier protocol that keep the ordering exact.

Universal block audio is enabled by default and can be disabled with `IE_AUDIO_BLOCK=0`. Every production sample ticker now opts into block ticking, and block length is bounded by the quiet span each ticker reports, so the block path is bit-identical to the per-sample path by construction rather than by inspection. Section 5 describes the span rule and the two passes it lets the pipeline hoist.

Alpha-blended Voodoo triangles are vectorised as far as the blend itself. The interpolation, clamp, alpha test, chroma key, fog and dither stages run in the SIMD kernel as for any other setup, and only the blend arithmetic runs per surviving lane, because it reads each target's own destination pixel and multiplies through per-target factors. That arithmetic lives in one shared function called by both the scalar rasteriser and the kernel, which is what makes the two bit-identical: gc's choice of FMA fusion for `src*srcFactor + dst*dstFactor` depends on the function the expression sits in, so two copies of the same source text drift by an ulp, and blending feeds the colour back through the factors where the truncating pack would otherwise have hidden it.

Bulk bus spans are enabled by default and can be disabled with `IE_BUS_SPANS=0`. Device paths that move whole files, ROM images and directory listings through the bus historically did so one byte at a time, which measured at 139 MB/s against 11 GB/s for the same transfer as a copy, and at 24 MB/s against 12 GB/s on a sparse backing. `MachineBus.ReadSpan` and `WriteSpan` take the bulk path only where it cannot be told apart from the loop it replaces: any span touching MMIO, any span the debug access service is observing, and any span not wholly inside either `bus.memory` or the backing falls back to the per-byte path. The switch changes speed and nothing else, which a differential test pins by running both settings.

Page dirty tracking is opt-in at bus construction with `IE_PAGE_DIRTY=1`, and IEMon also enables it lazily when whole-machine reverse history is first recorded unless `IE_MON_EPOCH_HISTORY=0`. It answers "which pages of guest RAM changed since I last looked" for several consumers at once without any of them destroying another's view: a global epoch counter plus a per-page generation that only ever increases, with nothing ever cleared. A consumer closes the current epoch before scanning and keeps its own cursor, so a page dirtied during a scan belongs to the epoch that scan opened rather than being lost or counted twice. With tracking inactive no tables are allocated, the write path pays one pointer test, and consumers receive an inactive cursor, which they are required to read as "assume everything is dirty" rather than as "nothing changed". Section 8 describes the writer and consumer protocols.

Epoch-driven reverse history is switched on automatically the first time whole-machine reverse history is recorded, and `IE_MON_EPOCH_HISTORY=0` is the kill switch that forces the legacy full-scan path. Arming is lazy so that a session which never reverse-debugs pays none of the page-dirty write-path cost. Without epoch capture the monitor's whole-machine reverse history scans all of guest RAM on every step to build a sparse page set and then diffs it against the previous capture; on a machine with gigabytes of RAM that scan dominates the step. With epoch capture on the monitor keeps a page-dirty cursor over the bus and a delta copies only the pages the guest wrote since the last capture. Full checkpoints still

take the complete scan, and draining the cursor at each checkpoint rebaselines the delta stream. The reconstruction is identical to the full-diff path: the cursor reports a page as dirty whenever it was written, so the epoch delta is a superset of the strict byte diff, and a page written back to all zeroes is emitted as a delete exactly as the diff would. A differential test reconstructs the same machine state from an epoch history and a legacy history driven by an identical write sequence. Section 8 describes the page-dirty cursor the capture rides on.

The IEScript compile cache is opt-in with `IE_SCRIPT_COMPILE_CACHE=1`. A parsed script proto does not depend on the Lua state it runs in. It is cached by script name plus exact source text and shared across the validate pass, the run, and later re-runs under the same name rather than being reparsed each time. Including the name keeps runtime diagnostics tied to the correct script. The cache only ever holds scripts that parse cleanly, so a syntax error is reported the same way with the cache on or off. IEScript also gains additive batching APIs that need no switch: `audio.write_regs` performs one Lua-to-Go call and applies each pair as an ordered bus write, matching a sequence of `audio.write_reg` calls exactly, and `sys.wait_until` evaluates a predicate immediately and then after at most the requested number of compositor frames.

SIMD acceleration kernels are enabled by default on amd64 builds and can be disabled with `IE_SIMD=0`. Hot pixel and sample spans (compositor blend and normalise, blitter fill and colour expand, software Voodoo untextured and blended spans, audio post-effects) dispatch through package-level function variables that point at bit-exact SIMD variants when the host provides the AVX2 baseline; every kernel keeps a scalar leaf as its canonical reference, and non-amd64 targets, hosts without the baseline, or `IE_SIMD=0` route the scalar path. The SIMD variants are additive and never alter emulated results.

Triple-Buffer Protocol

All video systems except VideoChip use a lock-free triple-buffer protocol for `GetFrame()`:

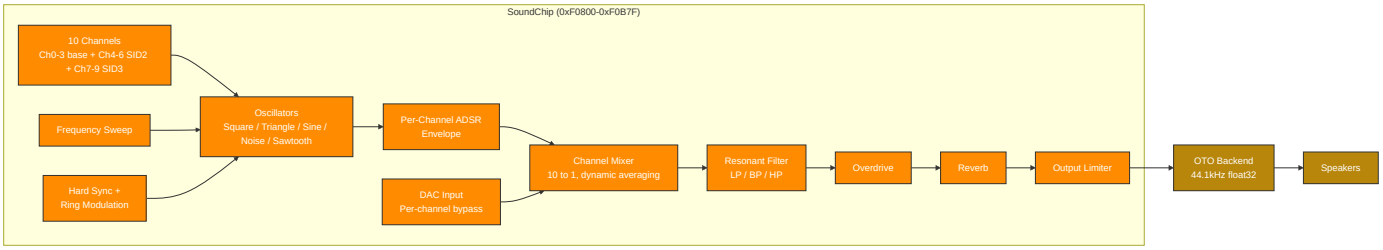
```
Slots:  writeIdx = 0 (producer-owned)
        sharedIdx = 1 (atomic, in-transit)
        readingIdx = 2 (consumer-owned)

Producer (render goroutine, after rendering into frameBufs[writeIdx]):
    writeIdx = sharedIdx.Swap(writeIdx)

Consumer (compositor, GetFrame):
    readIdx = sharedIdx.Swap(readIdx)
    return frameBufs[readIdx]
```

Important: `GetFrame()` performs an atomic Swap - calling it twice in a row swaps back to the previous buffer. Always call once and save the result.

On resolution change, all 3 buffer slots are reallocated and indices reset to `writeIdx=0`, `sharedIdx=1`, `readingIdx=2`.

5. Audio Subsystem

Synthesis Pipeline



SoundChip Dual-Interface Architecture

The SoundChip exposes two register interfaces for its channels:

Legacy per-waveform interface (`0xF0900-0xF09FF`) - dedicated register blocks with two decoded sweep-register exceptions:

Decoded ranges: `0xF0900-0xF093F except 0xF0914 and 0xF0918`, `0xF0940-0xF097F plus 0xF0914`, and `0xF0980-0xF09BF plus 0xF0918`.

Range	Channel	Default Waveform
`0xF0900-0xF093F`	Ch 0	Square, except `0xF0914` is triangle sweep and `0xF0918` is sine sweep
`0xF0940-0xF097F` plus `0xF0914`	Ch 1	Triangle
`0xF0980-0xF09BF` plus `0xF0918`	Ch 2	Sine
`0xF09C0-0xF09FF`	Ch 3	Noise
`0xF0A00-0xF0A6F`	-	Sawtooth + modulation/effects

Within the modulation/effects window, `0xF0A00-0xF0A1C` holds the sync-source and ring-modulation-source registers, `0xF0A20-0xF0A6F` holds the sawtooth legacy registers, and `0xF0A40`, `0xF0A50`, and `0xF0A54` hold overdrive and reverb controls.

FLEX unified interface (`0xF0A80-0xF0B7F`) - 4 channels with identical 64-byte register blocks. Each channel can be any waveform type:

Offset	Register	Description
`+0x00`	`FREQ`	16.8 fixed-point Hz (value / 256.0)
`+0x04`	`VOL`	Volume (0-255)
`+0x08`	`CTRL`	Enable (bit 0), gate (bit 1)
`+0x0C`	`DUTY`	Pulse width duty cycle; high byte is PWM depth scaled by 1/256
`+0x10`	`SWEEP`	Frequency sweep rate
`+0x14-0x20`	`ATK/DEC/SUS/REL`	ADSR envelope
`+0x24`	`WAVE_TYPE`	Waveform selection
`+0x28`	`PWM_CTRL`	Pulse width modulation
`+0x2C`	`NOISEMODE`	Noise mode (0=white, 1=periodic, 2=metallic, 3=psg)
`+0x30`	`PHASE`	Phase reset
`+0x34`	`RINGMOD`	Ring modulation (bit 7=enable, 0-2=source)
`+0x38`	`SYNC`	Hard sync (bit 7=enable, 0-2=source)
`+0x3C`	`DAC`	DAC mode bypass (signed 8-bit sample, -128=-1.0, +127=+1.0)

FLEX channels are at `FLEX_CH0_BASE = 0xF0A80`, stride = `0x40`. Primary channels 0-3 occupy `0xF0A80-0xF0B7F`; SID2 flex channels 4-6 occupy `0xF0C40-0xF0CFF`; SID3 flex channels 7-9 occupy `0xF0D40-0xF0DFF`. `AUDIO_CTRL` uses bit 0 for enable and bit 1 for freeze. `ENV_SHAPE` remains at `0xF0804` for channel 0, with per-channel shapes at `0xF0860 + channel*4`. Both legacy and FLEX interfaces write to the same underlying 10-channel mixer. The FLEX interface is preferred for new code - the legacy interface exists for backward compatibility.

Filter and Modulation

The SoundChip's global resonant filter (`0xF0820-0xF0830`) supports low-pass, band-pass, and high-pass modes with cutoff frequency, resonance, and optional filter modulation source/amount registers. Cutoff and resonance smooth toward register targets at the same 0.02 coefficient used by per-channel filters. SID 12-bit DAC quantization truncates to remove half-LSB DC bias.

Sample Generation Locking

`ReadSample` advances registered sample tickers before mixing the same sample. `GenerateSample` then takes `chip.mu` only for channel-state mutation, channel mixing, register-visible smoothing state, and a post-processing configuration snapshot. SID mixer options, overdrive, the global filter, reverb, and the master normaliser run after `chip.mu` is released. Their mutable DSP state is protected by `postFXMu`, which is also used by reset, debug snapshot, and restore.

`ReadSamples(dst)` is the block-rendering API used by the OTO backend. It is single-consumer: one pending-block state and one mixer-capture scratch are shared across calls, so two concurrent pulls would corrupt each other. Concurrent register writes from other goroutines are supported and expected; concurrent pulls are not. It installs a pending audio-block state for the duration of the pull. ReadSamples uses safe block ticking only when every active sample ticker implements `ReadSamplesBlockTicker`, SFX allows block ticking, and no sample mixers are registered. `IE_AUDIO_BLOCK=0` forces the per-sample path instead, which is retained as the oracle the block path is tested against. Otherwise it records each sample's ticker output and mixer contribution in order. Either way it flushes pending segments of up to 64 samples under one channel-mix lock. Any register write or SoundChip setter reached from a ticker first flushes the pending segment, so single-threaded `ReadSamples` output matches the `ReadSample` loop. Concurrent CPU writes remain bounded by the segment size and are allowed to apply earlier than a sample-at-a-time interleaving.

Block length is bounded by the quiet span the tickers report. An engine that mutates SoundChip state at sample positions it knows in advance implements `QuietSpanTicker`, and `QuietSamples` returns how many samples may pass before its next such write: the gap to the next register event for the event-driven engines, the gap to the next replay tick for AHX, the gap to the next envelope step for the PSG, and the gap to the end of the song for all of them. ReadSamples advances by the smallest span any registered ticker reports, so no register write ever falls inside a block and a block is exactly the sequence of single ticks it replaces. An engine that writes on every sample, which is the case for MOD, WAV and the AROS DMA shim, reports zero and renders in blocks of one sample; that is still the block path, so the per-sample fallback is reached only when a sample mixer is registered or SFX is sounding.

Two passes are hoisted out of the per-sample loop by that structure. The block-invariant half of the post-processing snapshot, meaning filter type and modulation depth, overdrive, reverb mix and comb decays, the SID mixer options and the master normaliser configuration, is captured once per flush, because none of it can move while `chip.mu` is held. The smoothed cutoff and resonance, the recurrent filter state and the modulation sample still advance per sample, in the same order as before. The output clamp is the only stateless pass in the pipeline and runs over the whole segment as one span, which is where the SIMD clamp kernel is wired.

Enabled channels that happen to be silent are still mixed. The channel mixer divides the sum by the number of enabled channels rather than the number of audible ones, so dropping a silent channel would change every other channel's contribution, and their oscillator phase has to advance regardless. Silent-channel specialisation is therefore excluded as a semantic change rather than deferred.

A guest register write and the renderer both want `chip.mu`, and the renderer holds it while it mixes a whole segment, so a CPU write waits for that mix. The event ring, enabled by default with `IE_AUDIO_EVENT_RING=0` as the kill switch, removes the wait: the guest bus path publishes the write into a ring and returns, and the renderer applies it at the end of the next flush, which is the same boundary the synchronous path got from flushing before it took the lock. Only the guest bus entry point publishes. Engine writes, setters, register reads, reset and shutdown stay synchronous, because they either run on the renderer itself or need the state they just wrote to be visible at once.

Those synchronous callers are barrier writers. Each one closes admission to the ring, waits for every producer that had already entered to publish or back out, flushes the pending block, drains the ring, and only then applies its own mutation, all under a barrier mutex so concurrent barrier writers are strictly serialised. A queued write can therefore never be applied after a write issued later, and a full ring is not a special case: the producer simply takes the barrier path, which is what it would have done with no ring at all. Reset and shutdown seal the ring, draining everything committed and leaving admission shut.

The ring's head and tail are stored by different goroutines, the guest bus path advancing head on every publish and the renderer advancing tail on every drained event. Left adjacent the two stores would contend for one cache line across the two goroutines. A contention benchmark measures that false sharing at roughly a 4.5 times penalty on the development host, so head and tail take separate cache lines, as do the publish and apply counters that mirror them. The padding changes no observable behaviour; it is layout only, has no kill switch, and a static test pins the separation so a later field reshuffle cannot fold two hot atomics back onto one line. This was the only structure a measurement pass found worth padding: the idle machine's other shared counters are either written by a single goroutine or updated far too rarely to share a line under contention.

The sample tap has its own lifetime rules, because a tap callback is guest-facing code the chip calls with its own state exposed. Each installed tap is held in its own record carrying a retired flag, and one chip-wide mutex is held across every callback, whichever record owns it. `SetSampleTap` and `ClearSampleTap` publish the replacement first and then retire the record they replaced, so an invoker that loaded the old record just before the store waits on that mutex and finds the record retired instead of calling into a tap the caller has already removed. On return no callback on the old tap is running and none can start.

The invocation mutex is chip-wide rather than per record because the guarantee spans generations. If the mutex were held per record, a setter that published a replacement while an earlier tap was still running would leave a second setter retiring an idle record and returning immediately, with that earlier tap still executing. One mutex for all generations makes every setter wait for whatever callback is in flight, whoever installed it.

That wait is unbounded on purpose. A timeout would let the setter return with the old callback still running, which is the guarantee the caller asked for, so there is nothing to trade it against.

A callback that replaces or removes its own tap cannot take that mutex, because its own invocation holds it. `SetSampleTapFromCallback` and `ClearSampleTapFromCallback` are the forms for that case: they shut the gate immediately, as the blocking forms do, but skip the wait for a callback in flight, which is the caller. Reentrancy is a choice of entry point rather than something the chip detects: calling a blocking setter from inside a callback waits on the caller's own invocation and blocks for good. Detecting it instead would mean recording the invoking goroutine, and resolving a goroutine identity costs microseconds per lookup, far more than generating the sample it would be charged against, so nothing on the sample path does it.

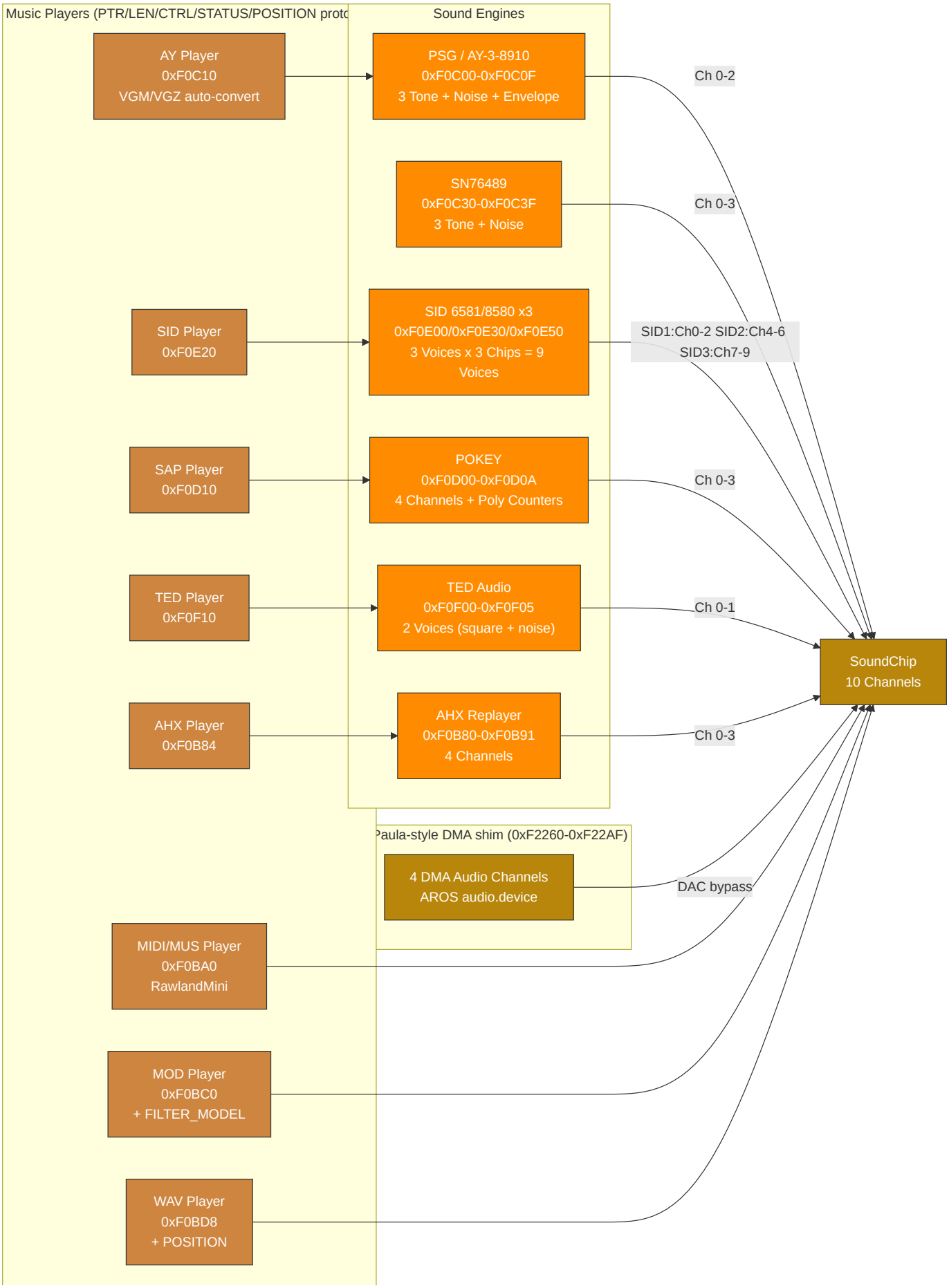
Barrier writers drain the ring themselves rather than asking the renderer to drain and waiting for an acknowledgement. Nothing pulling audio is a normal state, for a paused machine, a headless run or a build with no backend attached, and a writer that waited for a renderer drain would hang in all of them. The consumer role is instead serialised by `chip.mu`, which both the renderer and a barrier writer hold across a drain, so the tail cursor still has one writer at a time.

The ring is enabled by default. With `IE_AUDIO_EVENT_RING=0`, the synchronous path is unchanged. With the ring enabled, a register write issued while the renderer is mixing costs about 170 ns instead of about 7.4 µs on the measured development host.

The master normaliser separates setter-owned configuration from audio-thread-owned dynamics. Configuration setters publish a generation counter; the audio thread applies pending envelope and lookahead resets before the next normaliser step. Register-driven engine synchronisation can batch SoundChip writes through `HandleRegisterWrites`, preserving write order while taking the chip mutex once. Idle playback paths for SID, TED, MIDI, MOD, AHX, POKEY, and SFX

use atomic gates before taking their engine or channel mutexes. POKEY event playback drains current-sample events through a reusable scratch buffer prepared when events are loaded.

Engine and Player Routing


Music Players (PTR/LEN/CTRL/STATUS/POSITION proto
Sound Engines
AY Player
0xF0C10
VGM/VGZ auto-convert
PSG / AY-3-8910
0xF0C00-0xF0C0F
3 Tone + Noise + Envelope
Ch 0-2
SN76489
0xF0C30-0xF0C3F
3 Tone + Noise
Ch 0-3
SID Player
0xF0E20
SID 6581/8580 x3
0xF0E00/0xF0E30/0xF0E50
3 Voices x 3 Chips = 9
Voices
SID1:Ch0-2 SID2:Ch4-6
SID3:Ch7-9
SAP Player
0xF0D10
POKEY
0xF0D00-0xF0D0A
4 Channels + Poly Counters
Ch 0-3
TED Player
0xF0F10
TED Audio
0xF0F00-0xF0F05
2 Voices (square + noise)
Ch 0-1
SoundChip
10 Channels
AHX Player
0xF0B84
AHX Replayer
0xF0B80-0xF0B91
4 Channels
Ch 0-3
Paula-style DMA shim (0xF2260-0xF22AF)
4 DMA Audio Channels
AROS audio.device
DAC bypass
MIDI/MUS Player
0xF0BA0
RawlandMini
MOD Player
0xF0BC0
+ FILTER_MODEL
WAV Player
0xF0BD8
+ POSITION

Audio Engine Plus Enhanced Mode

All five classic-style sound engines have a "Plus" enhanced mode, activated by writing `1` to their respective `PLUS_CTRL` register:

Engine	PLUS_CTRL Address	Enhancements
PSG+	`0xF0C20`	Enhanced render path: oversampling, filtering, drive/room shaping, stereo voicing
SID+	`0xF0E19`	Enhanced render path for SID voices (oversampling/filter/drive/room shaping)
POKEY+	`0xF0D09`	Enhanced render path (oversampling/filter/drive/room shaping)
TED+	`0xF0F05`	Enhanced render path plus TED-specific response shaping
AHX+	`0xF0B80`	AHX voice-state mapping with stereo spread/panning and room processing

The PSG uses the AY/YM logarithmic 16-step volume curve by default. A legacy linear curve is retained for older material that depends on it.

When Plus mode is enabled, the engine retains full backward compatibility with the standard register set while exposing additional capabilities. AHX maps tracker state to native SoundChip channels instead of producing an AROS audio DMA stream; AHX+ uses a 64-sample crossfade when enabling/disabling to prevent audio glitches.

AROS Paula-Style DMA Shim ABI

The AROS audio block at `0xF2260-0xF22AF` is an Intuition Engine shim for AROS `audio.device`, not a full Amiga Paula implementation.

- Guest code writes the shim with 32-bit aligned writes (`MOVE.L`). Narrower writes are outside the current ABI.
- A `DMACON` channel transition from `0` to `1` latches that channel's pointer, length, period, and volume. Active playback uses the latched values until the buffer ends.
- If the latched period or length is zero at arm time, the channel is not activated. The channel `DMACON` bit is cleared, the status bit is set, and a level-3 interrupt is raised when the corresponding `INTENA` bit is set.
- Buffer exhaust sets the status bit, raises a level-3 interrupt when the corresponding `INTENA` bit is set, marks the channel inactive, and clears that channel's `DMACON` bit. The guest must write `DMACON` enable again to start the next buffer.
- Out-of-range pointers beyond the active AROS profile RAM deactivate the channel, mute the DAC output, set the status bit, and raise a level-3 interrupt when the corresponding `INTENA` bit is set.
- Pointer writes ignore bit 0, length is a word count and preserves odd values, period writes with zero are ignored, and volume writes are clamped to `0..64`.

M68K HostFS DOS Bridge ABI

The DOS block at `0xF2220-0xF225F` is the MMIO command bridge used by the AROS m68k-ie packet handler and by flat M68K programs. In flat M68K mode the DOS bridge is mapped directly, rooted at the runtime file directory, and connected to the monitor symbol table. Resetting or reloading flat M68K mode closes the current DOS bridge handles and locks, removes its MMIO mapping, clears runtime ownership, and installs one fresh bridge for the reloaded program. Stale host handles and duplicate mappings therefore do not survive program reload.

The guest writes up to four argument registers and then writes `AROS_DOS_CMD`; the host-backed device executes the request synchronously and returns AmigaDOS-style `RESULT1` and `RESULT2` values. `ADOS_CMD_EXAMINE_ALL` accelerates `ACTION_EXAMINE_ALL` through a 20-byte big-endian request descriptor, guest span validation, direct ExAllData packing, eac_LastKey continuation, and ERROR_ACTION_NOT_KNOWN fallback for match strings or hooks.

AROS HostFS fast paths use strict bulk guest-memory helpers, a 64 KiB sequential read-ahead cache, and cache invalidation on non-sequential reads, seeks outside cache, writes, truncates, close, create, delete, rename, and dirty close paths. Spans outside active guest RAM, high-pointer requests unsupported by the active bus, and low/high backing seam crossings fail closed. External host changes are best-effort and are not continuously watched.

Shared Host Socket ABI

The shared host socket block at `0xF2500-0xF257F` is mapped in every recognised CPU and operating-system mode except BASIC. It is available to IE64, IE32, M68K, x86, 6502, Z80, IntuitionOS, EmuTOS and AROS. EmuTOS and IntuitionOS expose the hardware mapping but do not yet provide guest-side socket integration. BASIC does not map this block. BASIC `HOST NET` configures host Wi-Fi only and does not enable guest sockets.

`0xF2400-0xF24FF` remains the SYSINFO ABI, so socket registers occupy the next 128-byte MMIO block. The shared host socket block uses 96-byte big-endian request descriptors, strict guest-memory helper copies, 64 KiB send/receive caps, 128-byte sockaddr caps, guest-visible descriptor handles, WaitSelect fd_set translation and Release/ReleaseCopy/Obtain transfer keys. The guest writes `REQ_PTR` and `REQ_LEN`, places the descriptor in guest RAM, and triggers synchronous dispatch by writing `CMD`. Each 32-bit register and descriptor word is big-endian. Byte writes are ordered from the most significant byte at offset zero to the least significant byte at offset three. A command assembled from byte writes dispatches only after all four command bytes have been written. Payload, fd_set, timeval, hostname, hostent and sockaddr buffers are copied through validated guest spans. The neutral names use the `HOST_SOCKET_*` prefix. Existing `AROS_HOST_SOCKET_*` names remain aliases with the same values.

Supported command values are `1=socket`, `2=bind`, `3=listen`, `4=accept`, `5=connect`, `6=sendto`, `7=recvfrom`, `8=shutdown`, `9=setsockopt`, `10=getsockopt`, `11=getsockname`, `12=getpeername`, `13=ioctl`, `14=close`, `15=WaitSelect`, `16=gethostbyname`, `17=gethostbyaddr`, `18=gethostname`, `19=Dup2`, `20=GetEvents`, `21=ReleaseSocket`, `22=ReleaseCopyOfSocket`, and `23=ObtainSocket`. The Unix backend supports IPv4 TCP and UDP sockets, makes host file descriptors non-blocking, hides them behind guest-visible handles, maps `WaitSelect` fd_sets through that handle table, and rejects arbitrary emulator process descriptors with `EBADF`. `ReleaseSocket` removes the caller's guest descriptor and stores the host descriptor under a release key. `ReleaseCopyOfSocket` stores a duplicated host descriptor and leaves the caller's descriptor active. Passing release key `-1` asks the backend to allocate a positive key; `ObtainSocket` consumes that key and returns a new guest descriptor, or fails with `EWOULDBLOCK` when the key is absent. Raw sockets, packet filters, route manipulation, interface configuration, and monitoring APIs return unsupported errors. When networking is disabled or no backend is present, the block fails closed with `ENOSYS`.

Linux and Darwin provide the Unix host-socket backend. Windows and other host platforms still expose the mapped register block in non-BASIC modes, but socket commands fail closed with `ENOSYS`.

Changing or reloading the guest mode first unmaps the old socket block, closes every active or released host descriptor exactly once, clears both guest handle tables, and then installs the replacement mapping when the new mode permits it. Reserved descriptor markers are not closed, and repeated cleanup is safe.

Subsong Selection

SID, SAP, and AHX players support subsong selection for multi-tune files. Each player has a subsong register that selects which tune to play from a multi-song file.

SID PSID playback captures CIA1 timer-A latch writes at `$DC04/$DC05`; when non-zero, the player uses that latch as `cyclesPerTick`, so multispeed tunes run at `clockHz / latch`. SID MMIO opts into wide-write fanout: 16-bit and 32-bit writes are decomposed into little-endian byte register writes.

Generic Live-MIDI Port (Shared Subsystem)

The live-MIDI port is a CPU-agnostic MMIO device that feeds a raw running-status MIDI byte stream straight into the synth, distinct from the file player (`SOUND PLAY "x.mid"`) but sharing the same `MIDIEngine`, the same pool of 10 MIDI voices, and the same sound-chip mix key. There is one synth and one runtime-status owner; the live port and the file player never double-mix.

Three byte-wide registers drive it: `IE_MIDI_LIVE_DATA` (`0xF0BF4`, write a raw MIDI byte), `IE_MIDI_LIVE_STATUS` (`0xF0BF5`, read bit 0 = live port active), and `IE_MIDI_LIVE_CTRL` (`0xF0BF6`, write bit 0 = reset / all notes off). `midi_live.go` is a running-status state machine that turns the byte stream into `MIDIEvents` (note-on/off, control change, program change, pitch bend, including messages split across multiple writes) and applies them to the shared engine. Because the device hangs off the MachineBus, every core and EhBASIC reach it directly; the EhBASIC `MIDI NOTE/PROG/CTRL/SEND/RESET` keywords emit onto these registers. Live notes take precedence over the file player at voice allocation while active (a live note steals a file voice before another live voice) and clear on a CTRL reset. Per-architecture symbol names are declared in each SDK include (`ie32/ie64/ie65/ie68/ie80/ie86.inc`), windowed to each core's address convention. The Ebiten runtime status bar's `MIDI` legend lights when either the file player is playing or the live port is active (`MIDIEngine.LiveActive()`), so both MIDI sources share one indicator.

EmuTOS Atari MIDI ACIA bridge

EmuTOS does not know IE's native live-MIDI register map; it talks MIDI only through the Atari ST MIDI port, an **MC6850 ACIA**. `atari_midi_acia.go` is a minimal, output-only MC6850 shim (`AtariMIDIACIA`) that bridges that ACIA to the same `LiveMIDI`/`MIDIEngine` path, so notes from ST software or GEMDOS `Bconout(3, ...)` reach IE's synth. It is wired **only in EmuTOS mode** (`main.go`, guarded by `modeEmuTOS`) and holds no synth or voice state of its own: control writes forward to the LiveMIDI parser, status reads always report `TDRE` (ready to transmit), and there is no MIDI-in. The shim maps two address forms of the Atari contract `$FFFC04` (RS=0: control/status) and `$FFFC06` (RS=1: TX data / RX): the bus-canonical low-16 alias `0x0000FC04/06` (which the sign-extended guest access `0xFFFFFC04/06` is normalized to before handler lookup) and the 24-bit Atari hardware alias `0x00FFFC04/06` (which does not pass through sign-extension normalization and must be mapped explicitly). Both aliases use `MapIONoShadow` because they fall inside guest RAM and status/data accesses must not mirror handler values into guest memory. A control write of `ACIA_CTRL_MASTER_RESET` (CR1:CR0 == 11) calls `LiveMIDI.Reset()`. The IKBD ACIA at `$FFFC00/02` is deliberately **not** mapped: keyboard already flows through the IOREC pump in `emutos_loader.go`. This bridge is EmuTOS-specific; there is no generic ACIA emulation for other modes.

6. Memory Map

All CPU cores observe the same guest physical address space. Address ranges in this table are not per-core private maps unless explicitly stated. Some ranges have multiple architectural uses: for example, a decoded RAM range may also contain conventional worker buffers, framebuffer memory, or OS/profile-owned allocations. In those cases, the table describes reservations within the shared physical map, not separate mappings.

The `Decoded as` column describes how the bus routes a physical access. The `Architectural use` and `Owner / lifetime` columns describe the guest contract or current subsystem reservation inside that decoded mapping. Overlapping rows are intentional when a reservation lives inside a broader shared-RAM range.

Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
`0x00000-0x9EFFF`	636KB	Shared RAM	Main low-memory programme/data area	All CPU cores	Guest and boot-profile convention	Includes IE32 vector table base at `0x00000` and programme load convention at `0x01000`.

Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
`0x9F000-0x9FFFF`	4KB	Shared RAM	Default stack seed region	All CPU cores	CPU reset convention	IE32 and IE64 seed stack pointers at `0x9F000`; it is still ordinary shared RAM.
`0xA0000-0xAFFFF`	64KB	VGA VRAM window	VGA graphics aperture	All CPU cores	VGA subsystem	x86 can also access this as conventional VGA graphics memory.
`0xB8000-0xBFFFF`	32KB	VGA text buffer	VGA text aperture	All CPU cores	VGA subsystem	Conventional text-memory range.
`0xD0000-0xDFFFF`	64KB	Voodoo texture memory	Voodoo texture upload/storage window	All CPU cores	Voodoo subsystem	Shared physical texture-memory aperture.
`0xF0000-0xF049B`	1180B	MMIO	VideoChip, copper, blitter, palette, and extended blitter registers	All CPU cores	Video subsystem	Layer-0 video control and DMA-style graphics operations.
`0xF0700-0xF07FF`	256B	MMIO	Terminal, serial, mouse, keyboard, and `RTC_EPOCH` (`0xF0750`)	All CPU cores	I/O subsystem	Input, terminal, and clock-facing low MMIO.
`0xF0800-0xF0B7F`	896B	MMIO	SoundChip channels, including FLEX	All CPU cores	Audio subsystem	Runtime audio-chip register block.
`0xF0B80-0xF0B91`	18B	MMIO	AHX engine/player	All CPU cores	Audio subsystem	Player-facing register block.
`0xF0BA0-0xF0BBF`	32B	MMIO	MIDI/MUS player	All CPU cores	Audio subsystem	Player-facing register block.
`0xF0BC0-0xF0BD7`	24B	MMIO	MOD player	All CPU cores	Audio subsystem	Player-facing register block.
`0xF0BD8-0xF0BF3`	28B	MMIO	WAV player	All CPU cores	Audio subsystem	Player-facing register block.
`0xF0BF4-0xF0BF6`	3B	MMIO	Generic live-MIDI port	All CPU cores	Audio subsystem	Byte-wide running-status MIDI stream: data write, active-status read, and reset control.
`0xF0C00-0xF0C0F`	16B	MMIO	PSG engine (AY-3-8910/YM2149 registers)	All CPU cores	Audio subsystem	Register-select/data-compatible PSG block.
`0xF0C10-0xF0C1F`	16B	MMIO	PSG / AY player	All CPU cores	Audio subsystem	Player-facing register block.
`0xF0C20`	1B	MMIO	PSG+ control	All CPU cores	Audio subsystem	Extended PSG control byte.
`0xF0C30-0xF0C3F`	16B	MMIO	Native SN76489 latch/data, ready, and LFSR mode registers	All CPU cores	Audio subsystem	Native SN76489-compatible control block.

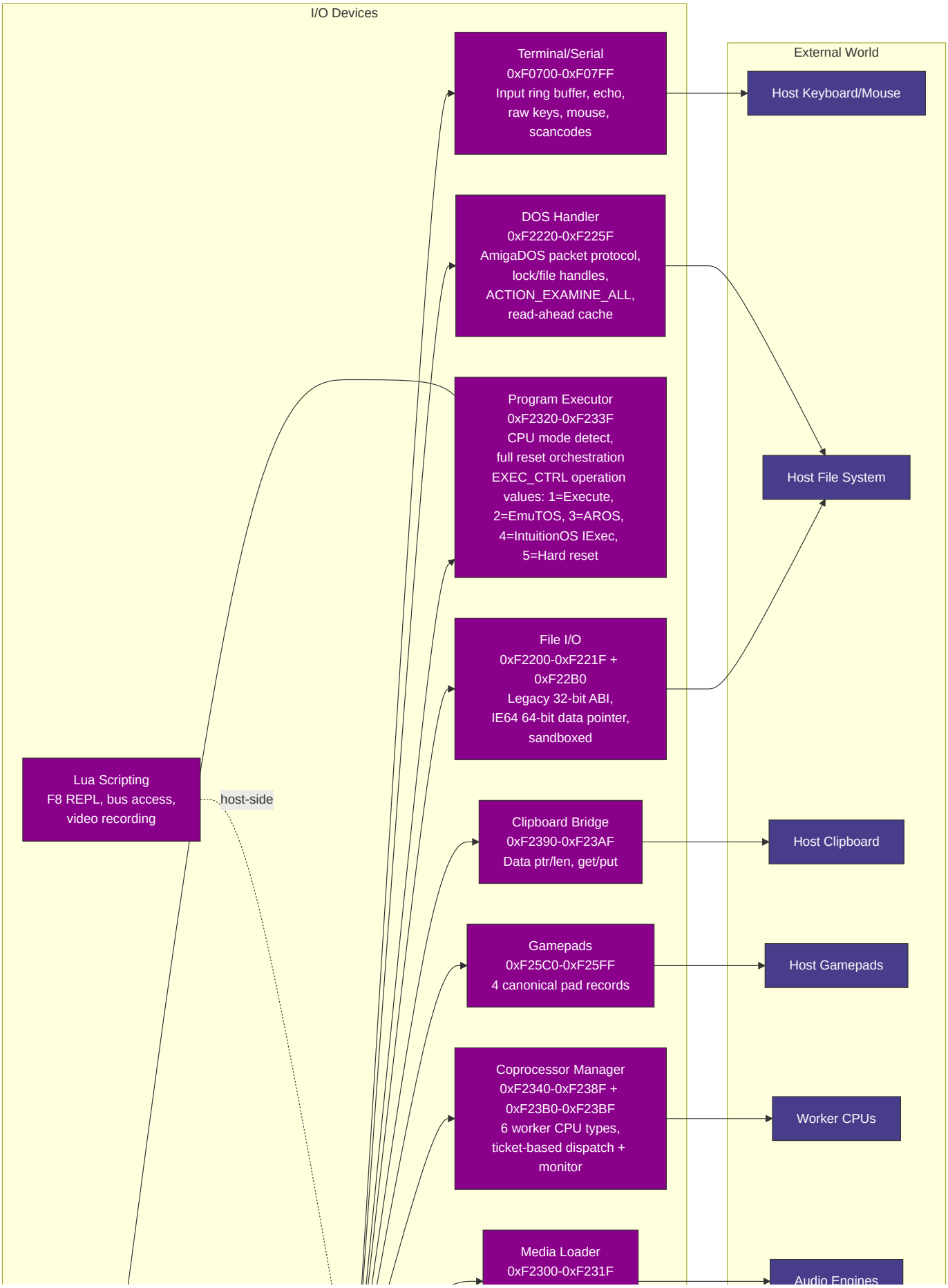
Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
0xF0C40-0xF0CFF	192B	MMIO	SID2 flex-style SoundChip channels 4-6	All CPU cores	Audio subsystem	Multi-SID/FLEX channel range.
0xF0D00-0xF0D0A	11B	MMIO	POKEY engine	All CPU cores	Audio subsystem	POKEY register block.
0xF0D10-0xF0D20	17B	MMIO	SAP player	All CPU cores	Audio subsystem	Player-facing register block.
0xF0D40-0xF0DFF	192B	MMIO	SID3 flex-style SoundChip channels 7-9	All CPU cores	Audio subsystem	Multi-SID/FLEX channel range.
0xF0E00-0xF0E1C	29B	MMIO	SID1 engine (6581/8580)	All CPU cores	Audio subsystem	Primary SID register block.
0xF0E20-0xF0E2D	14B	MMIO	SID player	All CPU cores	Audio subsystem	Player-facing register block.
0xF0E30-0xF0E4C	29B	MMIO	SID2 engine (6581/8580)	All CPU cores	Audio subsystem	Secondary SID register block.
0xF0E50-0xF0E6C	29B	MMIO	SID3 engine (6581/8580)	All CPU cores	Audio subsystem	Tertiary SID register block.
0xF0E80-0xF0EFF	128B	MMIO	SoundChip SFX trigger legacy aliases	All CPU cores	Audio subsystem	Channels 0-3 aliases for the trigger-oriented SFX range.
0xF2600-0xF29FF	1KB	MMIO	SoundChip SFX trigger extended channels	All CPU cores	Audio subsystem	Channels 0-31 trigger-oriented SFX range.
0xF0F00-0xF0F05	6B	MMIO	TED audio engine	All CPU cores	Audio subsystem	TED audio registers.
0xF0F10-0xF0F1F	16B	MMIO	TED player	All CPU cores	Audio subsystem	Player-facing register block.
0xF0F20-0xF0F6B	76B	MMIO	TED video	All CPU cores	TED video subsystem	Stride-4 TED video register mapping.
0xF3000-0xF6FFF	16KB	MMIO / TED VRAM aperture	TED private video RAM	All CPU cores	TED video subsystem	Bus-routed TED character, colour, and screen memory.
0xF1000-0xF13FF	1KB	MMIO	VGA registers	All CPU cores	VGA subsystem	Sequencer, CRTC, graphics-controller, attribute-controller, and DAC state.
0xF1400-0xF140F	16B	MMIO	Host helper	All CPU cores	Host-helper subsystem	Security-sensitive command helper gated by runtime configuration.
0xF2000-0xF2017	24B	MMIO	ULA registers	All CPU cores	ULA subsystem	Includes paged VRAM data port.
0xFA000-0xFBAFF	6912B	ULA VRAM aperture	ULA display memory	All CPU cores	ULA subsystem	Separate decoded aperture for ULA VRAM access.

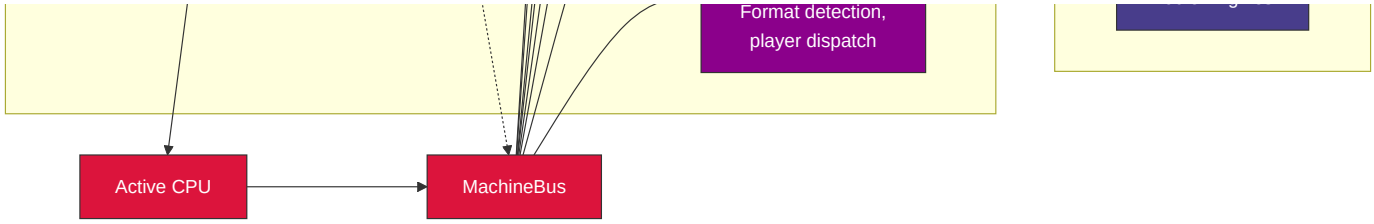
Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
0xF2100-0xF213F	64B	MMIO	ANTIC registers	All CPU cores	ANTIC subsystem	ANTIC display-list and DMA control block.
0xF2140-0xF21FB	188B	MMIO	GTIA registers	All CPU cores	GTIA subsystem	GTIA colour, player/missile, priority, and collision state.
0xF2200-0xF221F	32B	MMIO	File I/O	All CPU cores	File-I/O bridge	Legacy host-file bridge register block with 32-bit name, data, length, status, error, and read-cap registers.
0xF22B0-0xF22B7	8B	MMIO64	File I/O IE64 extension	IE64	File-I/O bridge	FILE_DATA_PTR64, a 64-bit data-buffer pointer for IE64 code that reads or writes host files from high guest RAM. The legacy File I/O block remains unchanged.
0xF2220-0xF225F	64B	MMIO	M68K HostFS DOS bridge	All CPU cores while mapped	AROS or flat M68K mode	Host-backed AmigaDOS-style command block. Flat M68K reset/reload removes the old mapping and installs one fresh bridge.
0xF2260-0xF22AF	80B	MMIO	AROS Paula-style DMA shim	All CPU cores	AROS profile	Audio/DMA compatibility shim.
0xF2300-0xF231F	32B	MMIO	Media loader	All CPU cores	Media-loader subsystem	Format-agnostic media loading control block.
0xF2320-0xF233F	32B	MMIO	Program executor	All CPU cores	Program executor	CPU/program switching control block.
0xF2340-0xF238F	80B	MMIO	Coprocessor manager	All CPU cores	Coprocessor subsystem	Worker lifecycle, dispatch, and ticket registers.
0xF2390-0xF23AF	32B	MMIO	Clipboard bridge	All CPU cores	Clipboard bridge	Host clipboard integration register block.
0xF23B0-0xF23BF	16B	MMIO	Coprocessor extended monitor registers	All CPU cores	Coprocessor subsystem	Per-worker monitor/status extension.
0xF23C0-0xF23DF	32B	MMIO	AROS IRQ diagnostic registers	AROS M68K profile while AROS loader is active	AROS profile	Read-only diagnostic MMIO mapped by the AROS loader and unmapped during AROS DMA teardown; not a universal interrupt-controller ABI.
0xF23E0-0xF23FF	32B	MMIO	Bootstrap HostFS	All CPU cores	Bootstrap profile	HostFS boot helper register block.
0xF2400-0xF24FF	256B	MMIO	SYSINFO RAM-size discovery	All CPU cores	System information ABI	Reports total and active visible RAM.
0xF2500-0xF257F	128B	MMIO	Shared host socket bridge	All CPU cores while mapped	Every recognised non-BASIC mode	Host-backed socket command bridge. EmuTOS and IntuitionOS have no guest-side integration yet.

Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
0xF2580-0xF259F	32B	MMIO	CPU wait service	All CPU cores	Timing/wait ABI	CPU wait writes park until the next VBlank edge or a latched RTC_MONO_USEC deadline, capped by a 50 ms safety timeout; reads return 0.
0xF25A0-0xF25BF	32B	MMIO	Coprocessor instance discovery	32-bit-addressing CPU cores and main-CPU BASIC	Coprocessor subsystem	Selected type's instance limit, selected-instance state, mailbox layout version, worker window and ring addresses, plus an atomic all-instance liveness mask.
0xF25C0-0xF25FF	64B	MMIO	Gamepad discovery	All CPU cores	Gamepad ABI	Read-only canonical status, buttons, and stick axes for four pads. Non-headless Ebiten builds, including js/wasm browser builds, publish once per frame; headless builds report no pads. Writes are ignored.
0xF8000-0xF87FF	2KB	MMIO	Voodoo 3D registers and palette	All CPU cores	Voodoo subsystem	3D control, state, and palette register block.
0xF8140-0xF823F	256B	MMIO	Voodoo fog table	All CPU cores	Voodoo subsystem	64 entries x 4 bytes.
0x100000-0x5FFFFF	5MB	Shared RAM / VRAM-backed region	Main video framebuffer and graphics-visible memory	All CPU cores	Video subsystem plus guest convention	Subranges may be reserved for coprocessor worker buffers.
0x200000-0x27FFFF	512KB	Shared RAM	IE32 worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x280000-0x2FFFFF	512KB	Shared RAM	M68K worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x300000-0x30FFFF	64KB	Shared RAM	6502 worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x310000-0x31FFFF	64KB	Shared RAM	Z80 worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x320000-0x39FFFF	512KB	Shared RAM	x86 worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x3A0000-0x41FFFF	512KB	Shared RAM	IE64 worker area	All CPU cores	Coprocessor convention	Lies inside the graphics-visible shared-memory range; not a separate address space.
0x420000-0x49FFFF	512KB	Shared RAM	M68K worker instance 1 area	All CPU cores	Coprocessor convention	Second M68K worker RAM; ordinary big-endian worker memory inside the graphics-visible shared-memory range.

Range	Size	Decoded as	Architectural use	Visible to	Owner / lifetime	Notes
`0x4A0000-0x51FFFF`	512KB	Shared RAM	x86 worker instance 1 area	All CPU cores	Coprocessor convention	Second x86 worker RAM inside the graphics-visible shared-memory range.
`0x520000-0x59FFFF`	512KB	Shared RAM	IE64 worker instance 1 area	All CPU cores	Coprocessor convention	Second IE64 worker RAM inside the graphics-visible shared-memory range.
`0x790000-0x792FFF`	12KB	Shared RAM	Coprocessor mailbox ring buffers	All CPU cores	Coprocessor subsystem	Twelve ring slots at a `0x400` stride; shared request/completion rings outside the main graphics-visible range.
`0x800000-0x80FFFF`	64KB	Shared RAM	Media-loader staging buffer	All CPU cores	Media-loader subsystem	Reservation at the base of the AROS fast-memory range when that profile is active.
`0x800000-0x1DFFFFF`	22MB	Shared RAM	AROS fast memory	All CPU cores	AROS profile	Profile-owned allocation range, not a distinct per-core map.
`0x1E00000-0x5DFFFFF`	64MB	Shared RAM / video-profile memory	AROS video RAM	All CPU cores	AROS profile / video subsystem	Profile-owned video allocation range within shared physical memory.

7. I/O Peripherals


I/O Devices
External World
Terminal/Serial
0xF0700-0xF07FF
Input ring buffer, echo,
raw keys, mouse,
scancodes
Host Keyboard/Mouse
DOS Handler
0xF2220-0xF225F
AmigaDOS packet protocol,
lock/file handles,
ACTION_EXAMINE_ALL,
read-ahead cache
Program Executor
0xF2320-0xF233F
CPU mode detect,
full reset orchestration
EXEC_CTRL operation
values: 1=Execute,
2=EmuTOS, 3=AROS,
4=IntuitionOS IExec,
5=Hard reset
Host File System
File I/O
0xF2200-0xF221F +
0xF22B0
Legacy 32-bit ABI,
IE64 64-bit data pointer,
sandboxed
Lua Scripting
F8 REPL, bus access,
video recording
host-side
Clipboard Bridge
0xF2390-0xF23AF
Data ptr/len, get/put
Host Clipboard
Gamepads
0xF25C0-0xF25FF
4 canonical pad records
Host Gamepads
Coprocessor Manager
0xF2340-0xF238F +
0xF23B0-0xF23BF
6 worker CPU types,
ticket-based dispatch +
monitor
Worker CPUs
Media Loader
0xF2300-0xF231F
Audio Engines



Gamepad MMIO

The read-only gamepad block occupies `0xF25C0-0xF25FF`. Non-headless Ebiten builds poll standard-layout controllers before overlay early returns and publish one coherent snapshot per frame. This includes native desktop and js/wasm browser builds; the browser path obtains controller state through the browser Gamepad API. Controller-specific layouts are normalised by the host mapping database. Headless builds publish an empty snapshot. Disconnecting a pad clears its previous buttons and axes. Stores are accepted but ignored.

`GAMEPAD_STATUS` reports the connected mask in bits 3:0 and connected-pad count in bits 11:8; four 12-byte pad records contain canonical buttons and packed signed-16 left/right stick axes.

Address	Width	Meaning
`0xF25C0` (`GAMEPAD_STATUS`)	32-bit	Connected mask in bits 3:0 and connected-pad count in bits 11:8.
`0xF25D0 + p*0x0C`	32-bit	Pad `p` buttons, for `p=0..3`.
`0xF25D4 + p*0x0C`	32-bit	Left X in bits 15:0 and left Y in bits 31:16.
`0xF25D8 + p*0x0C`	32-bit	Right X in bits 15:0 and right Y in bits 31:16.

Stick halves are signed 16-bit values. Host axes are clamped to `-1..1`, NaN becomes zero, and finite values are multiplied by 32767; right and down are positive. Byte and halfword gamepad reads use the addressed little-endian lane of the containing 32-bit word. This includes `JOY_HOME` in the third button byte and Y in the high axis half.

Bit	Name	Bit	Name
0	`JOY_UP`	1	`JOY_DOWN`
2	`JOY_LEFT`	3	`JOY_RIGHT`
4	`JOY_A`	5	`JOY_B`
6	`JOY_X`	7	`JOY_Y`
8	`JOY_LB`	9	`JOY_RB`
10	`JOY_LT`	11	`JOY_RT`
12	`JOY_SELECT`	13	`JOY_START`
14	`JOY_L3`	15	`JOY_R3`
16	`JOY_HOME`		

`JOY_LT` and `JOY_RT` are digital button states; the ABI has no analogue trigger fields. IE64, IE32, M68K, and x86 access the canonical addresses directly. 6502 and Z80 select extended bank `0x79` and use the Bank 1 window at `0x25C0-0x25FF`; their SDK includes provide `SET_GAMEPAD_BANK` and resolve the same register and button constants. EmuTOS and AROS can read the canonical M68K block directly, but adapting it to their conventional joystick APIs requires guest-side drivers and is not part of this MMIO contract.

The File I/O ABI keeps the original 32-bit register block at `0xF2200-0xF221F` for every CPU. IE64 adds `FILE_DATA_PTR64` at `0xF22B0` as a separate 64-bit MMIO register for data buffers in high guest RAM, including buffers

allocated from the native AOT arena. Non-IE64 software does not need to know about the extension.

Coprocessor Worker Dispatch

The coprocessor manager supports 6 worker CPU types (IE32, IE64, 6502, M68K, Z80, x86) with ticket-based job dispatch and mailbox ring buffers at `0x790000`. Each worker type has a dedicated shared-memory reservation in the unified physical map (see memory map above), not a private per-core address space. The main CPU enqueues work via MMIO writes; workers execute independently and post results back through their mailbox slots. Each ring has 16 descriptor slots but uses one slot to distinguish full from empty, so it can hold 15 queued requests at once. When `COPROC_IRQ_CTRL` bit 0 is set and an M68K IRQ target plus completion watcher have been configured, the coprocessor asserts M68K interrupt level 6 on job completion, with the finished ticket ID readable from `COPROC_COMPLETED_TICKET`. Non-M68K modes should observe completion through `POLL` or `WAIT`.

`COPROC_INSTANCE` selects a worker instance together with `COPROC_CPU_TYPE`. The JIT-capable worker types (M68K, x86, IE64) support instances 0 and 1; IE32, 6502, and Z80 support instance 0 only. Instance-1 windows are M68K `0x420000-0x49FFFF`, x86 `0x4A0000-0x51FFFF`, and IE64 `0x520000-0x59FFFF`, each 512 KiB. The mailbox holds twelve ring slots at a uniform `0x400` stride, and the ring index is `cpuTypeIndex*2 + instance` with no special case (the `0x400` stride exceeds a ring's `0x308` content, so no ring's last slot overflows into its neighbour). `COPROC_SELECTED_STATE` reports whether the selected worker is running, while `COPROC_INSTANCE_STATE` reports all live instances atomically without changing `COPROC_CPU_TYPE` or `COPROC_INSTANCE`; the aggregate `COPROC_WORKER_STATE` sets a per-type bit when any instance of that type is running. Start, stop, enqueue, ring depth, uptime, and ticket ownership are scoped to the selected instance. A worker must echo the mailbox layout version (`COPROC_MAILBOX_VERSION`) into its ring header at startup or the manager refuses to route it work and START fails with `COPROC_ERR_STALE_WORKER`.

The coprocessor discovery block at `0xF25A0-0xF25BF` reports the selected type's instance limit, selected-instance state, mailbox layout version, worker window, worker ring, and an atomic all-instance liveness mask. The mailbox contains twelve `0x400`-byte ring slots from `0x790000` through `0x792FFF`; each ring publishes layout version 1 and a worker must acknowledge that version before START succeeds.

Instance-1 windows are M68K `0x420000-0x49FFFF`, x86 `0x4A0000-0x51FFFF`, and IE64 `0x520000-0x59FFFF`, each 512 KiB; the ring index is `cpuTypeIndex*2+instance`, `COPROC_SELECTED_STATE` reports the selected worker, and `COPROC_INSTANCE_STATE` reports all live instances without changing the selectors.

M68K workers initialise their own stack bounds and supervisor/user stack pointers and arm the M68K JIT. Coprocessor shared-window pages route through memory helpers so their byte-swap rules match interpreted execution; both M68K worker RAM windows are excluded from that shared-window treatment and remain ordinary big-endian worker RAM. M68K block compilation is serialised across the main CPU and worker instances.

Lua Scripting

The Lua scripting engine (`script_engine.go`) runs in its own goroutine and provides host-side automation over the shared 32-bit bus/MMIO surface plus CPU-adapter debugger APIs. Its `mem.*` helpers are raw 32-bit bus helpers, not an above-4GiB IE64 RAM or CPU-virtual-address API. It supports an F8 REPL for interactive debugging, video recording, and direct chip register manipulation. Scripts use the `.ies` extension and are loaded via the IE Script Engine.

Recording Pipeline

Video recording sends compositor frames and 44.1 kHz mono audio to FFmpeg on separate pipes. The audio tap observes the mixed output without advancing the sound engines a second time. Recording follows wall-clock time; after an encoder stall the recorder discards missed video-frame debt and matching oldest buffered audio while preserving the newest output batch. This prevents a blocked encoder from replaying stale media in an unbounded catch-up burst after it resumes. Video

and audio recording pumps are independent; frozen or unchanged video is held, and audio starvation beyond 500 ms produces silence instead of stalling.

IEMon Reverse-Debug Snapshot Contract

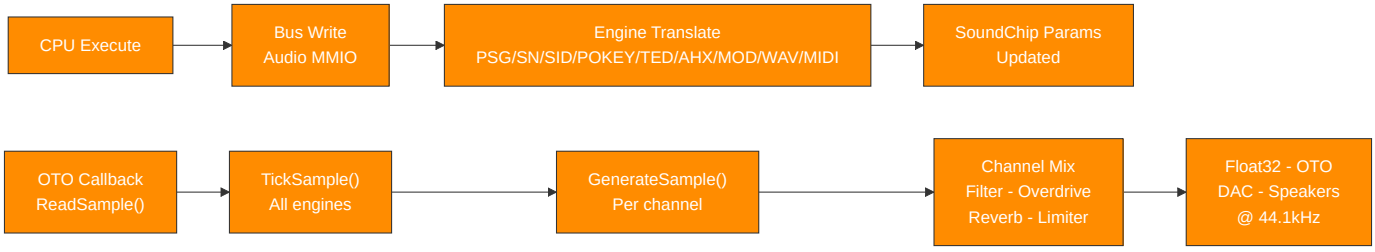
IEMon whole-machine snapshots enumerate the monitor CPU registry by stable CPU id and label, so profiles with omitted CPUs or multiple coprocessors restore by identity rather than by CPU type. Shared bus RAM and IE64 backing memory are stored as sparse 4 KiB pages. The reverse-history chain keeps full sparse checkpoints plus page deltas anchored to retained checkpoints; the monitor materialises deltas before `rg` or `rt` applies a restore.

Mutable devices join the snapshot contract through `MachineMonitor.RegisterSnapshotDevice`. Each device supplies a stable name, a version, and an opaque guest-visible state blob. Restore fails closed if a captured device is absent, preventing partial reverse-debug restores. The production monitor registers the main video chip, sound chip, terminal MMIO, command-style host helpers, compatibility audio/video engines, and AROS clipboard/audio-DMA bridges when present; new guest-visible timers, DMA engines, IRQ controllers, and host bridges must register their own versioned blobs before they are considered covered by whole-machine reverse debugging.

8. Data Flow

Video Pipeline

Audio Pipeline



CPU Mode Switching

Bus Access Filtering and Bulk Spans

Every guest access asks two questions before it touches memory: is there MMIO on this page, and is the debug access service watching. The first is answered by the I/O page bitmap and, once execution starts, by a sealed map snapshot that turns an atomic pointer load into a plain field read. Sealing is a lifecycle decision rather than a performance knob: a guest reset leaves the seal and the map alone, because the devices are still mapped and the map is still valid, while a reload unseals first, remaps, and reseals when the next runner starts. Remapping a sealed bus is a programming error and panics.

The second question used to be expensive. A single watchpoint anywhere makes the debug service active, and every access then took a mutex and scanned the guard and watch lists to reach an answer that was almost always no, which measured at 47 ns against 7 ns for an unwatched non-I/O read. A conservative per-page filter now answers the cheap half without the

mutex. It is one-sided by construction: a clear bit means no guard and no watchpoint covers the page and history is off, so the access cannot hit anything; a set bit means only "maybe", and the full scan runs exactly as before, so guard scope, CPU identity, access kind and the precise address range are all still decided by the original code. The filter is rebuilt whole whenever the guard set, the watch set or the history setting changes, and it is deliberately not an index of watchpoints. History recording widens it to every page, because history observes every access regardless of coverage.

Bulk transfers ask both questions once for a whole span instead of once per byte. `ReadSpan` and `WriteSpan` take the bulk path only when nothing could make an individual byte observable, and fall back to the per-byte path otherwise. That fallback is what keeps the fast path honest rather than a corner to be tidied away: watchpoints, access history and MMIO side effects are all preserved by refusing the span, not by trying to reproduce them in bulk.

A call site whose per-byte path used the fault-checked accessors carries a further obligation, because being byte-for-byte identical is not sufficient there: the span must not reach anything those accessors would have refused. `Read8WithFault` and `Write8WithFault` only ever touch `bus.memory`, rejecting every address at or above its end even when the backing would serve it, and they fault on an unmapped address inside a strict MMIO window. Those sites use `spanFaultCheckedEligible`, which narrows the general test to that window, so a guest buffer that used to be rejected keeps being rejected rather than quietly succeeding against backing memory. GEMDOS `Fread` and `Fwrite` are the current example.

Page Dirty Epochs

Page dirty tracking exists because several consumers want the same answer from guest RAM and a shared flag cannot give it to them. A flag answers "since somebody last looked", so whichever consumer reads it first robs the others.

A global epoch counter and a per-page generation replace it. A writer reads the epoch, and if the page generation already equals it there is nothing to do; that is the steady state and costs two atomic loads and a compare. Otherwise it raises the generation to the epoch with a compare-and-swap that only ever increases the value, so a slow writer carrying a stale epoch can never pull a page backwards, and then re-reads the epoch. If the epoch moved while it was publishing, it goes round again. That retry closes the one race the design exists to avoid: a writer that read epoch E, was pre-empted while a consumer closed E, and then published E into a page the consumer had already scanned. The retry republishes into the open epoch, where the next scan finds it.

A consumer closes the current epoch by advancing the counter from E to E+1, scans for pages whose generation lies in (lastSeen, E], and then moves its own cursor to E. Writes landing during the scan publish E+1 and surface next time. Nothing is ever cleared, so consumers cannot interfere with each other, and each holds its own cursor.

Scan cost is bounded by grouping pages into chunks that carry the maximum generation beneath them, raised by the same monotonic protocol. A chunk at or below the cursor cannot hold anything the consumer has not seen, so the scan skips it whole rather than walking every page of a multi-gigabyte machine.

Publication rides on the notification every guest RAM write already makes for M68K JIT invalidation, so a new write path cannot forget it without also forgetting invalidation. Bulk transfers publish one range rather than touching per-page state per byte. Publications for addresses past the tracked span are counted rather than ignored, because a tracker sized smaller than the RAM it covers would otherwise lose writes in silence.

A consumer that copies a dirty page must do so at a defined quiescent point, not merely after the scan reports it: within a closed epoch the page is known to have changed, but nothing stops it changing again while the copy runs. Whole-machine snapshots use the existing snapshot pause point. Any future per-page consumer must name its own point, which is why reverse-history capture is rewired to epochs separately rather than as part of the tracking mechanism.

9. Concurrency Model and System Timing

Goroutine Inventory

Goroutine	Clock Source	Rate	Synchronisation
CPU execution	Free-running `for running.Load()` loop	As fast as Go scheduler allows	MMIO writes invoke I/O callbacks under per-chip `mu`
VideoChip refresh	`time.NewTicker`	60Hz	VBlank timing, copper advancement, and blitter IRQ latency
VGA render	`time.NewTicker`	60Hz	Renders into triple-buffer slot, publishes via `sharedIdx.Swap()`
ULA render	`time.NewTicker`	60Hz	Same triple-buffer pattern
TED render	`time.NewTicker`	60Hz	Same triple-buffer pattern
ANTIC render	`time.NewTicker`	60Hz	Same triple-buffer pattern
Compositor	`time.NewTicker`	60Hz	Calls `GetFrame()` (atomic swap) on each source, blends, sends to display
OTO audio thread	Host audio hardware callback	44.1kHz	Calls `SoundChip.ReadSample()` -> `GenerateSample()` directly
Terminal output	`time.NewTicker`	100Hz	Flushes output buffer to host terminal
Coprocessor workers	On-demand	Varies	Independent CPUs with mailbox ring buffers
Script engine	Event-driven	Varies	Lua goroutine with frame-sync channel

Three Independent Clocks

```
CPU:   Free-running (no fixed clock -- instruction throughput varies with host speed)
Video: 6 video sources plus compositor tick at 60Hz; non-VideoChip sources use lock-free triple-
buffer handoff
Audio: OTO hardware callback drives sample generation at 44.1kHz -- no IE-owned goroutine
```

Synchronisation Model

Hot paths (lock-free): - `atomic.Bool` - CPU `running`, chip `enabled`, `compositorManaged` - `atomic.Int32` - triple-buffer `sharedIdx` - `atomic.Pointer` - selected CPU pointer

Cold paths (mutex-protected): - `sync.Mutex` (`chip.mu`, `video.mu`) - configuration changes, setup, stop

CPU-to-Video: - CPU writes to VRAM/MMIO invoke I/O callbacks under per-chip `mu` - Video chips read VRAM lock-free (snapshot-render pattern: copy under lock, render without lock)

CPU-to-Audio: - CPU writes to audio MMIO update channel parameters under `chip.mu` - OTO callback reads parameters atomically or under `chip.mu` for `GenerateSample()`

No CPU-video vsync coupling: - CPU runs ahead freely - no wait-for-vblank blocking - WAIT register polls `vblankActive atomic.Bool` without blocking the CPU

Appendix: Key Source Files

File	Role
registers.go	Master I/O address map - all region boundaries
machine_bus.go	MachineBus: autodetected guest RAM, MapIO, ioPageBitmap, Read/Write, SYSINFO accessors
machine_bus_span.go	ReadSpan/WriteSpan bulk transfers and the eligibility rules that send a span back to the per-byte path (IE_BUS_SPANS)
machine_bus_page_dirty.go	Epoch page dirty tracking: writer publication, consumer cursors, chunk-summary scans (IE_PAGE_DIRTY)
debug_access_pagefilter.go	Conservative per-page pre-filter over guards and watchpoints, rebuilt whenever either set changes
memory_sizing.go	Boot-time guest RAM autodetection: total guest RAM + active visible RAM with platform reserves
profile_bounds.go	Source-owned profile bounds for EmuTOS, AROS, IE64 BASIC
emulator_cpu.go	EmulatorCPU interface definition
video_interface.go	VideoSource, VideoOutput, ScanlineAware interfaces
video_compositor.go	Compositor pipeline, Z-order blending
video_chip.go	VideoChip + Copper + Blitter + Palette
video_vga.go	VGA engine (Sequencer, CRTC, GC, AC, DAC)
video_ula.go	ULA engine (Spectrum display)
video_ted.go	TED video engine
video_antic.go	ANTIC + GTIA engines
video_voodoo.go	Voodoo 3D pipeline
audio_chip.go	SoundChip (10-channel synthesis)
psg_engine.go	PSG / AY-3-8910
sn76489_chip.go	SN76489 programmable sound generator
sid_engine.go	SID 6581/8580
pokey_engine.go	POKEY
ted_engine.go	TED audio
ahx_engine.go	AHX replayer
mod_player.go	MOD player
wav_player.go	WAV player
midi_player.go / midi_engine.go / midi_parser.go	SMF .mid/.midi and Doom .mus; fixed RawlandMini IE SoundChip GM-style/chiptune interpretation, 10 active MIDI voices with deterministic stealing
midi_live.go	Generic live-MIDI MMIO port: running-status byte-stream parser feeding the shared MIDIEngine (all cores + BASIC)

File	Role
atari_midi_acia.go	EmuTOS-only MC6850 MIDI ACIA shim ($FFFC04/06, low-16 + 24-bit aliases) bridging output-only MIDI bytes into the shared LiveMIDI/MIDIEngine
cpu_ie32.go	IE32 CPU
cpu_ie64.go	IE64 CPU + interpreter
fpu_ie64.go	IE64 FPU (16 x float32)
jit_emit_arm64.go	IE64 JIT ARM64 emitter
jit_emit_amd64.go	IE64 JIT x86-64 emitter
cpu_m68k.go	M68K 68020
fpu_m68881.go	M68881 FPU (8 x float64)
cpu_z80.go	Z80
cpu_z80_runner.go	Z80 bus adapter, port I/O bridge, bank windows
cpu_six5go2.go	6502 + bus adapter, ioTable dispatch, bank windows
cpu_x86.go	x86
cpu_x86_runner.go	x86 bus adapter, shared port I/O bridge, bank windows, and VGA VRAM window
fpu_x87.go	x87 FPU (8 x float64 stack)
terminal_io.go	Terminal / Serial / Mouse / Keyboard
file_io.go	File I/O
program_executor.go	Program Executor
coprocessor_manager.go	Coprocessor Manager
clipboard_bridge.go	Clipboard Bridge
media_loader.go	Media format detection + player dispatch
script_engine.go	Lua scripting engine
aros_loader.go	AROS ROM boot manager
aros_dos_intercept.go	AmigaDOS packet handler (MMIO)
aros_audio_dma.go	AROS Paula-style DMA shim